國立中正大學

資訊工程研究所碩士論文

利用 40 奈米製程實現帶有提前退出機制的輕量級卷積神經網路於無人飛行載具火災偵測之應用

Implementation of Lightweight Convolutional Neural Networks with Early Exit Mechanism Utilizing 40nm CMOS Process for Fire Detection in Unmanned Aerial Vehicles

研 究 生 ： 張宸銘

指導教授 ： 鍾菁哲 博士

中華民國 一一二 年 八 月

國立中正大學碩士學位論文考試審定書

資訊工程學系

研究生 張宸銘 所提之論文

利用40奈米製程實現帶有提前退出機制的輕量級卷積神經網路於無人飛行載具火災偵測之應用

經本委員會審查，符合 碩 士學位論文標準。

學位考試委員會
召集人 李順裕 簽章

委員 鍾聖杰 李順裕 黃崇員力 盛鏐

指導教授 鍾聖杰 簽章

中華民國 112 年 7 月 26 日

摘要

近年來，邊緣運算的發展已成為未來科技的趨勢之一，並且被廣泛應用在各種產業中。無人機做為邊緣裝置的熱門選擇，在航拍、物流、農業、公共安全等方面皆獲得良好的成績。火災的檢測尤其是一個具有巨大潛力的應用場景，特別是在偏遠或難以到達的地區。在無人機上實現機器學習的技術能夠針對當前環境進行檢測，判斷是否有火災發生的徵兆，能夠節省人力資源的開支並提升判斷的準確度。

本論文根據森林火災的空拍圖像來進行火災的檢測，並且即時對火災發生的徵兆做出反應，以及早應對，避免森林火災的發生對於自然環境、人類居住區域等造成嚴重損壞。為了以低成本的硬體實現即時森林火災檢測，本論文使用 DoReFa-Net 的方法量化網路參數以及對激勵函數的輸出進行定點化，以減少記憶體的大小及計算量，並且引入提前退出的機制，從而實現低功率與低成本之卷積神經網路。

本論文所提出之帶有提前退出機制的卷積神經網路硬體加速器於 TSMC 40nm CMOS 製程下實現，並使用電源門控技術將閒置狀態的記憶體之電源關閉，達成低功耗的目標。本論文所提出之電路在硬體描述階段，對於火災場景的偵測最高可達到 81.49%的準確度，經由自動布局繞線實現後，所提出之硬體工作頻率最高可達 300MHz，功耗為 117mW。

關鍵字：無人機、森林火災偵測、DoReFa-Net、提前退出機制、卷積神經網路、低功耗晶片

Abstract

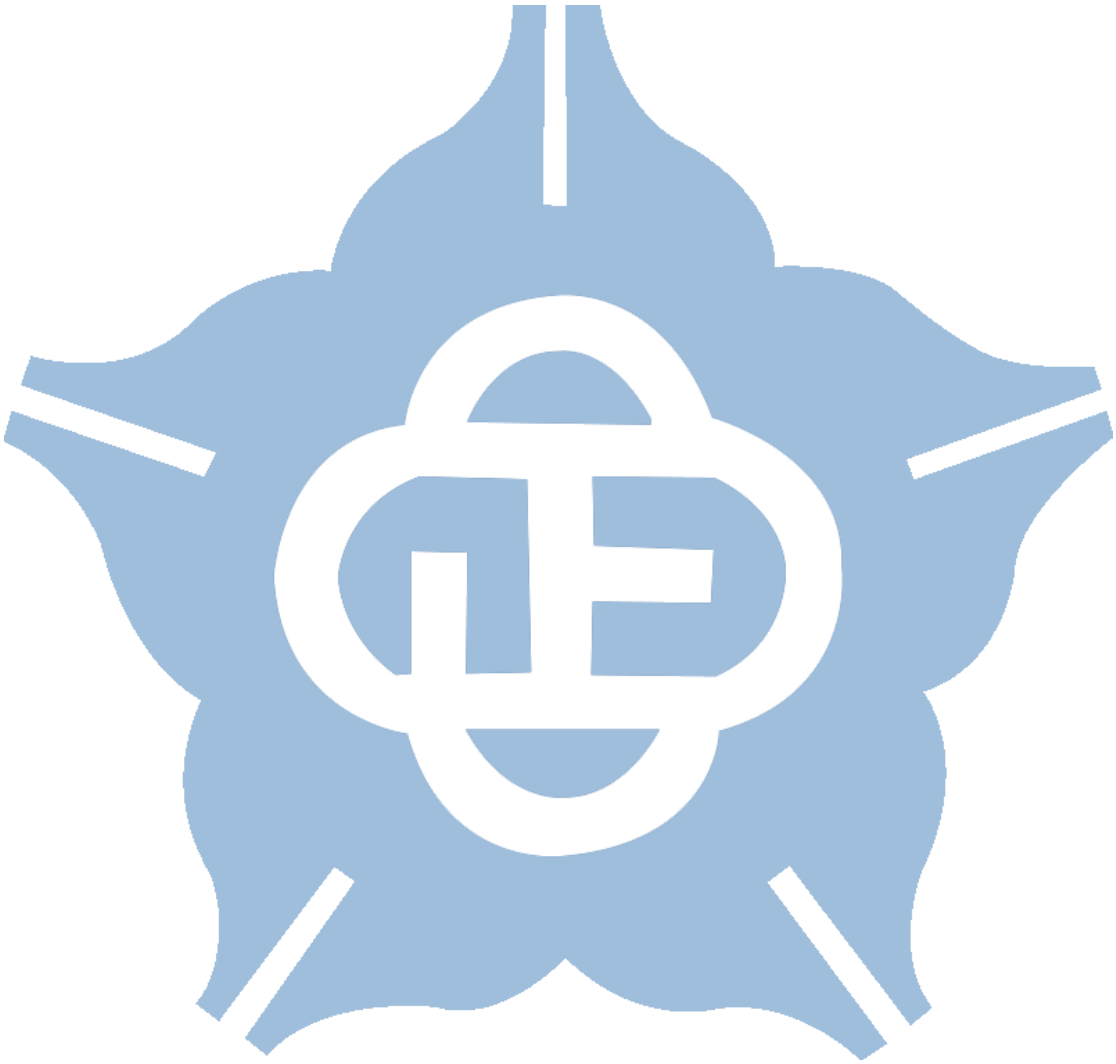
In recent years, edge computing has become one of the trends in future technology and has been widely applied in various industries. As popular edge devices, unmanned aerial vehicles (UAVs) have shown promising results in aerial photography, logistics, agriculture, public safety, and more. Fire detection, in particular, is a significant application with immense potential, especially in remote or hard-to-reach areas. Implementing machine learning on UAVs enables real-time fire detection, allowing for prompt responses and preventing severe damage to forests, natural environments, and human settlements caused by wildfires. This paper focuses on using aerial images of forest fires for fire detection and responding to fire signs in real time to mitigate the impact of forest fires.

To achieve real-time forest fire detection with cost-effective hardware, this paper adopts the DoReFa-Net method to quantize network parameters and apply fixed-point quantization to activation functions, reducing memory size and computational complexity. An early-exit mechanism is also introduced to enable a low-power and low-cost convolutional neural network.

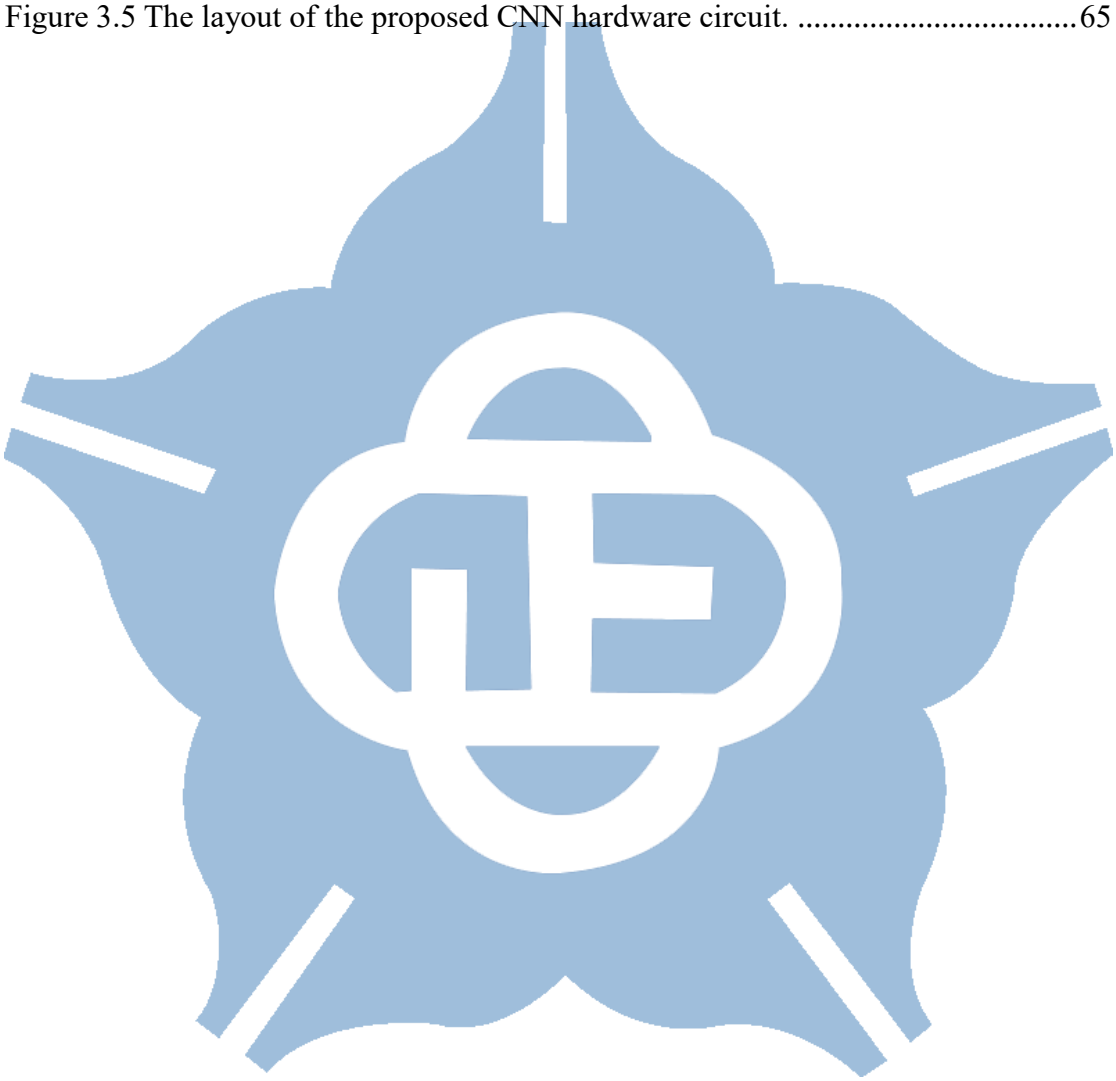
The proposed hardware accelerator for the convolutional neural network with the early-exit mechanism is implemented in TSMC 40nm CMOS process. The power-gating technique is employed to turn off the power of idle memory, achieving low power consumption. The hardware circuit achieves up to 81.49% accuracy for fire detection in fire scenarios during the hardware description phase. After automatic layout and routing, the proposed hardware operates at a maximum frequency of 300MHz with a power consumption of 117mW.

Keywords: Unmanned aerial vehicles (UAVs), forest fire detection, DoReFa-Net, early-exit mechanism, convolutional neural network, low-power chip.

Content



List of Figures



List of Tables

Chapter1 Introduction

1.1 Introduction to Unmanned Aerial Vehicles and Their Applications

In recent years, the crucial role of Unmanned Aerial Vehicles (UAVs) in the field of edge computing has been underscored. Despite their diminutive size, these lightweight devices are adept at performing various tasks in diverse climatic and geographical conditions. Consequently, their broad-based utility spans areas like agricultural surveillance, managing traffic, environmental observation, and search and rescue missions. Under the conventional cloud computing paradigm, the terminal device and the cloud necessitate data transfer, which then undergoes processing in the cloud. This mode of transmission, however, poses several challenges, including network delay, securing data, and preserving privacy. UAVs address these issues by performing autonomous computations and processing, paving the way for innovative application technologies.

Figure 1.1 Sample images from the VisDrone dataset[1].

Nonetheless, the advancement of UAV technology also brings forth several

hurdles. As mentioned in [1], the authors bring attention to the issue of UAV landing, during which ground conditions must be taken into account by the UAV. Incorrect operations could potentially lead to accidents involving pedestrians. Thus, they suggest an approach for crowd detection. They utilize the dataset referenced in [2], rich in imagery procured via UAVs, as shown in Figure 1.1. By identifying whether images contain crowds, they categorize crowds and employ a fully Convolutional Neural Network (CNN) to maintain lightweight operation. The structure of this process is illustrated in Figure 1.2.

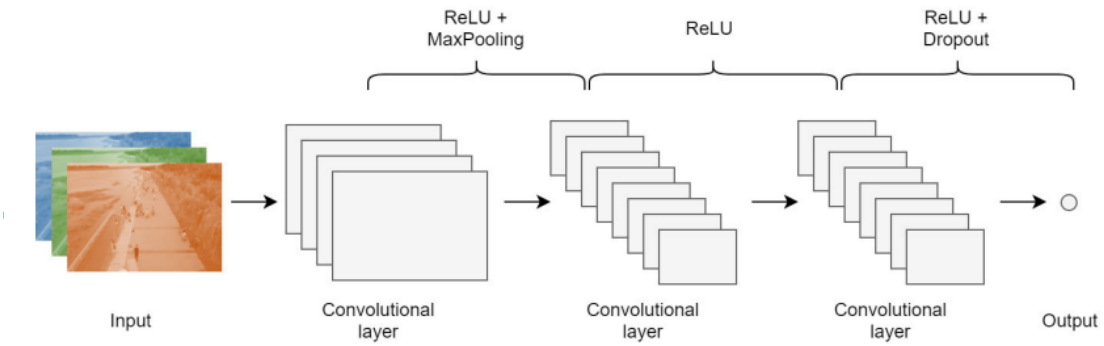


Figure 1.2 The fully convolutional neural network proposed in [2].

UAVs find extensive use in the domain of environmental surveillance. For instance, [3] outlines a technique for the preliminary detection of forest fires. Such wildfires wreak havoc on natural habitats, including forests, farms, and broader ecosystems. The authors created an algorithm specifically designed for detecting and monitoring forest fires. This technique capitalizes on the unique color traits of flames to identify and track areas where fire outbreaks occur, as depicted in Figure 1.3.

Further, [4] suggests a method incorporating environmental sensing technology. In their research, multiple elements, such as temperature, CO, and CO_2 concentrations, are monitored using sensors to detect potential fire signs. Likewise, [5] proposes a methodology to assess a fire's state based on the smoke's conditions.

In [6], they introduce a fire-specific dataset collected exclusively by drones, as

shown in Figure 1.4. This dataset comprises fire-related images and video footage captured by various drones in Northern Arizona, covering diverse angles, zoom levels, and camera varieties. This dataset bears a significant value in studying fires during their initial stages.

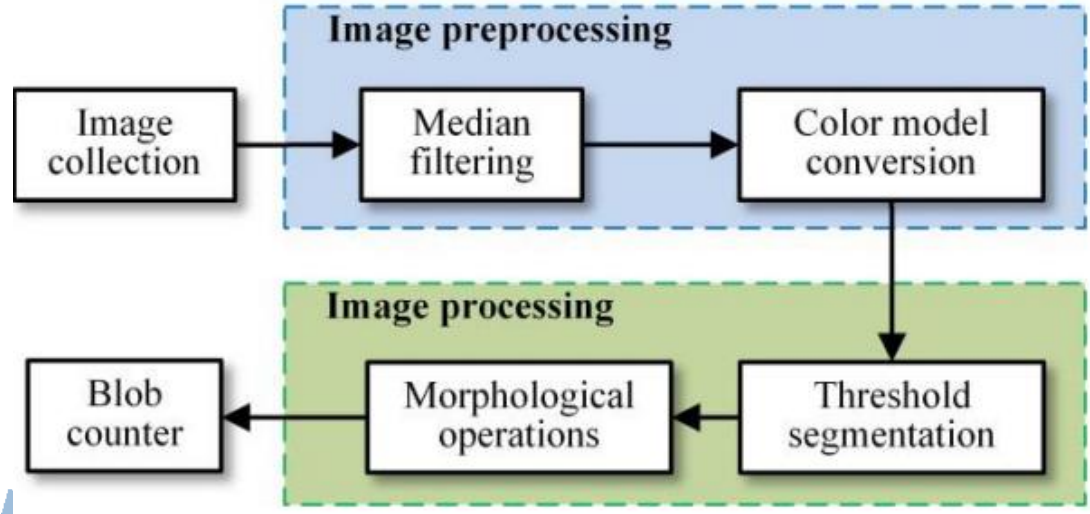


Figure 1.3 Flowchart of fire detection and tracking algorithm[3].

Figure 1.4 Sample images of pile burn[6].

In forest management, pile burning is commonly employed to dispose of forest residues, such as branches and leaves. This pile-burning process necessitates professional oversight for several days to avert uncontrolled fire incidents. Therefore, automated aerial surveillance systems could substantially mitigate the monitoring burden. In their study, the authors deploy Google's Xception model [7] as the

categorization framework for distinguishing between fire and non-fire zones, as depicted in Figure 1.5. This approach results in 96.79%, 94.31%, and 76.23% accuracy for the training, validation, and testing datasets.

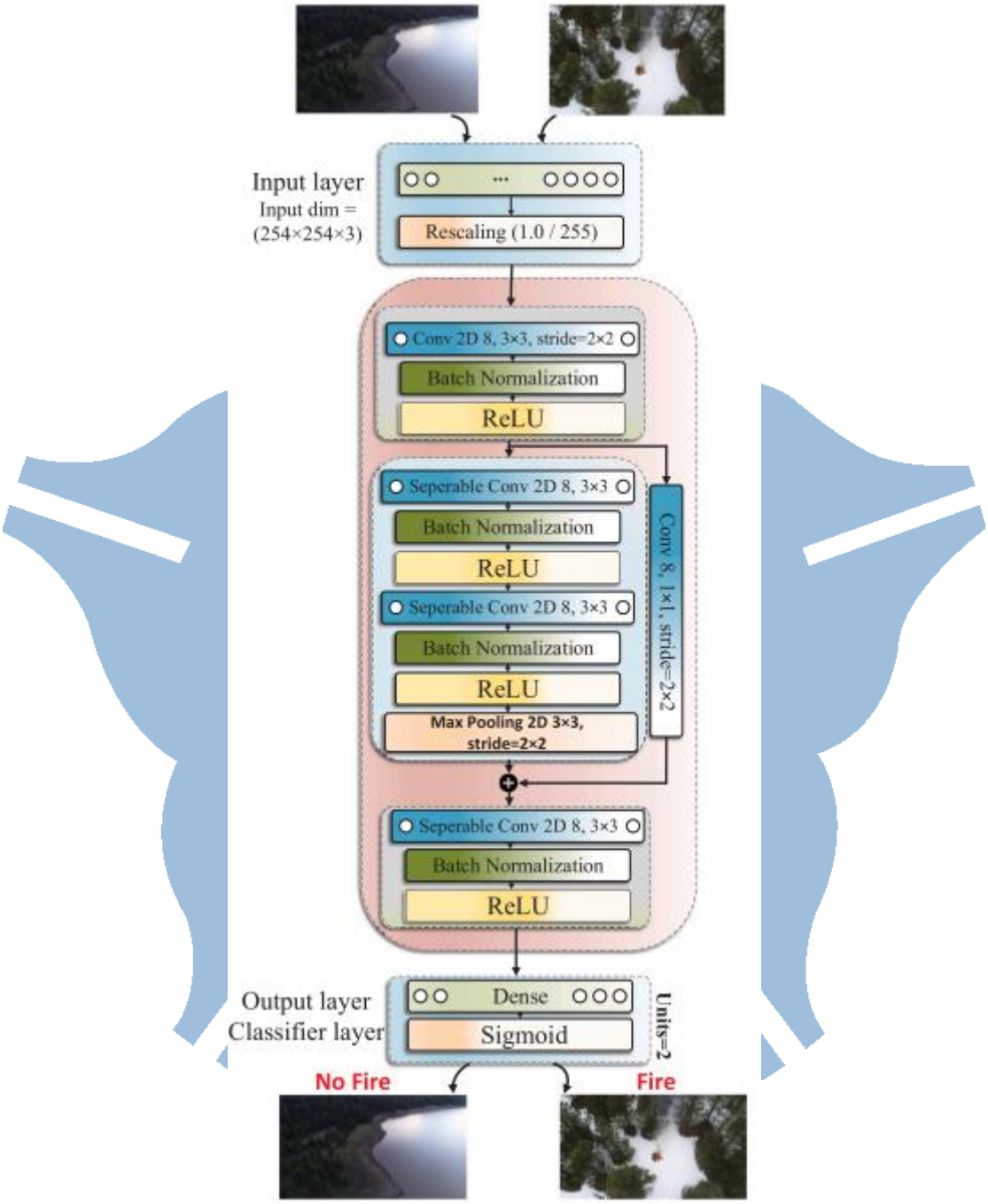


Figure 1.5 The Xception architecture[6].

In [8], the authors present an innovative classification structure for a wildfire identification dataset. This structure combines the EfficientNet-B5 [9] and DenseNet-

201 [10] networks to extract image features for classifying wildfires, illustrated in Figure 1.6. The model achieves an accuracy of 85.12% on the test set. However, the amount of the parameters for EfficientNet-B5 and DenseNet presents a roadblock for drone applications, thereby curbing their practical usability despite achieving a commendable level of accuracy.

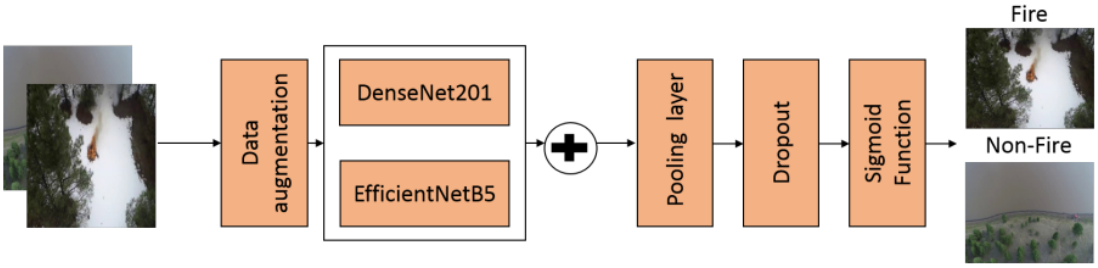


Figure 1.6 The classification framework mentioned in [8].

In [11], the authors proposed a transfer learning-based FT-ResNet50 model. This approach transfers the ResNet network and parameters, initially trained on the ImageNet dataset, to the forest fire dataset. The ResNet is refined using the Adam optimizer and Mish function, with optimization further enhanced by the Focal loss function. This architecture yielded an accuracy of 79.48%. Despite the fact that transfer learning with fine-tuning can augment image classification performance, an accuracy rate of merely 79.48% might not suffice for accurate image classification.

In the context of edge computing, drones are bound by their hardware capabilities, hindering the attainment of high computational levels. Consequently, selecting an appropriate strategy based on the application becomes crucial. Our goal should be to curtail hardware prerequisites to the greatest extent feasible without compromising functionality, thereby accomplishing a lightweight operation with minimal power consumption.

1.2 Introduction to Convolution Neural Network

A Convolutional Neural Network (CNN) is a deep learning model with convolutional layers. The convolutional operation, shown in Figure 1.7, is predominantly employed in fields such as image recognition, signal processing, and natural language processing. It embodies the traits of local perception and weight sharing, thereby equipping the network to learn from and extract features of the input images.

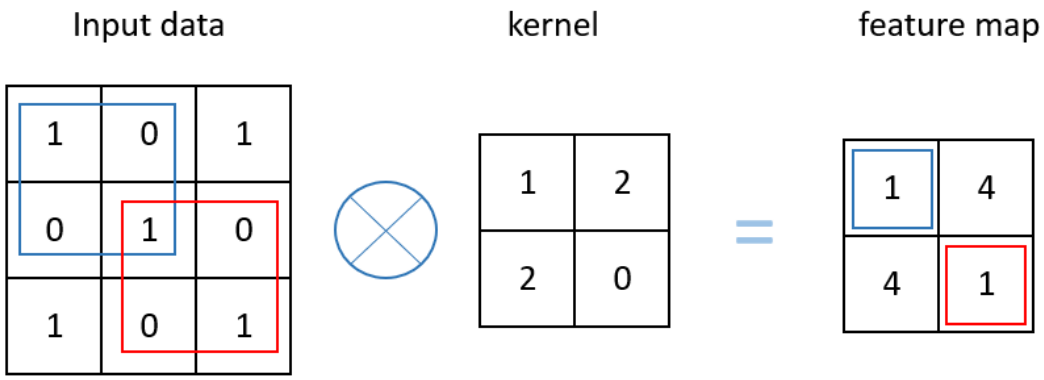


Figure 1.7 Example of the convolution operation.

The foundational architecture of a CNN encompasses an input layer, a convolutional layer, an activation function layer, a pooling layer, a fully connected layer, and an output layer. The input layer accepts input data, such as image data. Acting as the heart of a CNN, the convolutional layer undertakes the extraction of features from the input data by moving a convolutional kernel over it and subsequently creating a feature map. The activation function infuses non-linearity, enabling the CNN to comprehend more complex features. Common activation functions include ReLU and Sigmoid, with their transformations denoted in Equations 1.1 and 1.2, respectively. The pooling layer diminishes the feature map's size and decreases computational complexity, as shown in Figure 1.8. The fully connected layer converts the feature map, drawn out

by the convolutional layer, into a one-dimensional vector and conducts feature fusion via weighted summation. The responsibility of the output layer lies in predicting the output result.

$$ReLU(x) = max(0, x) \tag{1.1}$$

$$Sigmoid(x) = \frac{1}{1+e^{-x}} \tag{1.2}$$

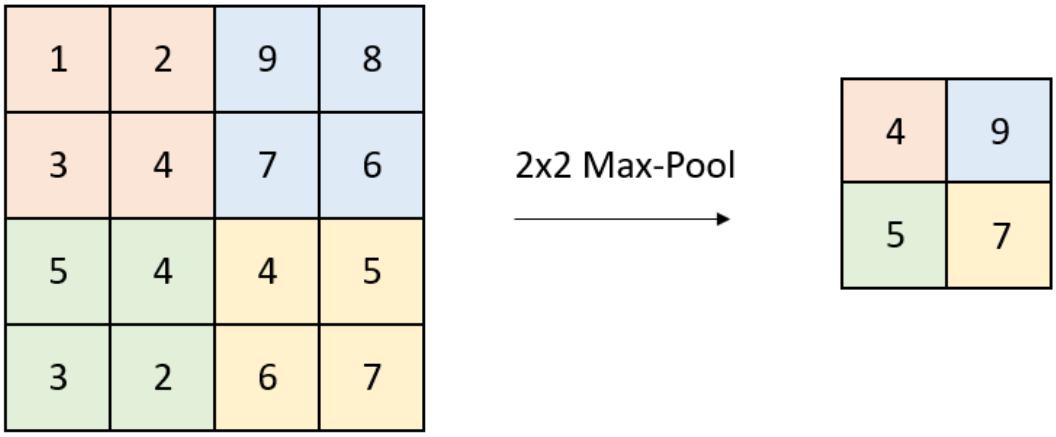
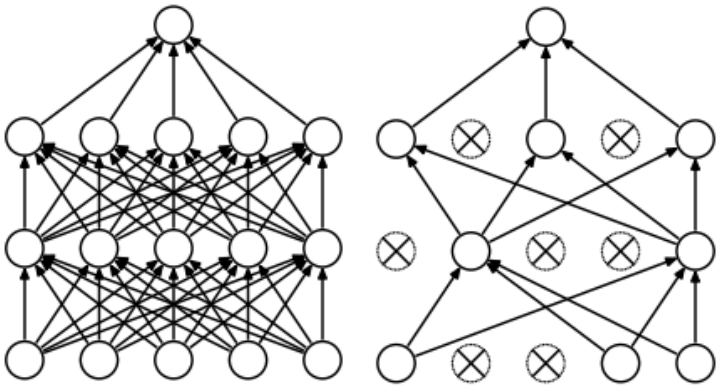


Figure 1.8 Example of a 2×2 max pooling.

In addition to the layers mentioned, additional techniques can be incorporated into neural network models to improve their performance. For instance, Dropout, a regularization method, was proposed by Nitish Srivastava and his team in 2014 [12]. This technique sporadically renders some neuron outputs to zero during the training phase, enhancing the model's stability and capacity for generalization, as illustrated in Figure 1.9.

Sergey Ioffe and his team introduced Batch Normalization in 2015 [13], a technique that enhances the speed and stability of training in neural networks. Its primary goal is to address the issue of internal covariate shift during the training process by normalizing each input to a standard normal distribution with a mean of 0 and a standard deviation of 1. By standardizing the disparate data in this manner, the

technique can mitigate the issue of gradient vanishing and expedite convergence.



(a)Standard neural new (b)After applying Dropout

Figure 1.9 Dropout net model

Equations 1.3 to 1.6 describe the batch normalization process, where *x* represents the inputs over a mini-batch. Initially, the mean and variance of the mini-batch are computed, as illustrated in Equations 1.3 and 1.4. Subsequently, the inputs are normalized based on the previously computed mean and variance, as depicted in Equation 1.5. Lastly, two training parameters, γand β, facilitate the model's feature distribution learning, as shown in Equation 1.6.

$$\mu B \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \tag{1.3}$$

$$\alpha_\beta^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_\beta)^2 \tag{1.4}$$

$$\hat{x_i} \leftarrow \frac{x_i - \mu_\beta}{\sqrt{\alpha_\beta^2 + \epsilon}} \tag{1.5}$$

$$y_i \leftarrow \gamma \hat{x_i} + \beta \equiv BN_{\gamma,\beta}(x_i) \tag{1.6}$$

Global Average Pooling (GAP) [14] is a widely adopted technique in convolutional neural networks that decreases the spatial dimensionality of feature maps. The fundamental concept of GAP entails conducting global average pooling on every feature map, which transforms its spatial dimensionality from H×W (height × width) to 1×1. This lessens the requirement for parameters and computations and contributes to

mitigating the risk of overfitting.

One of the most well-known CNN structures is LeNet [15], proposed by Yann Lecun and his team in 1998. This architecture incorporates two convolutional layers, pooling, fully connected, and Gaussian connection layers. Initially crafted to recognize handwritten numerals, LeNet is regarded as the ancestor of convolutional neural networks. The architecture is depicted in Figure 1.10.

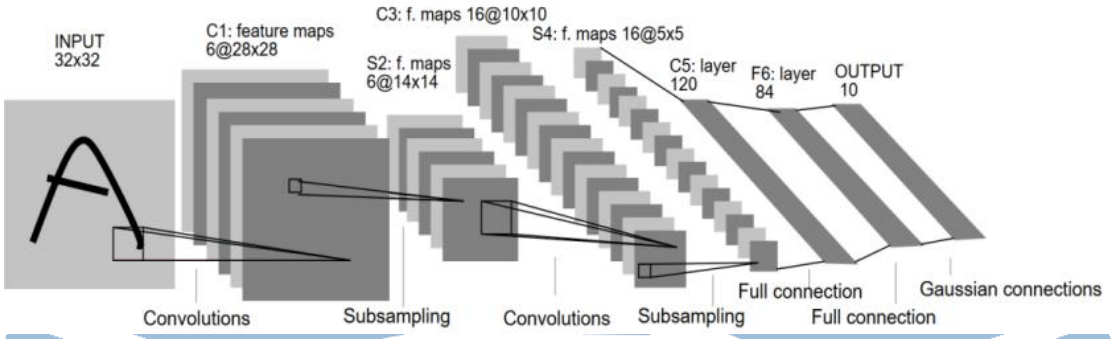


Figure 1.10 The architecture of LeNet-5 CNN used for digit recognition [15].

AlexNet [16], proposed by Alex Krizhevsky and his colleagues, emerged as the champion of the 2012 ImageNet competition. Introducing ReLU and Dropout techniques in AlexNet contributed to a highly impressive accuracy rate, leading to widespread ReLU adoption. The architectural design is exhibited in Figure 1.11.

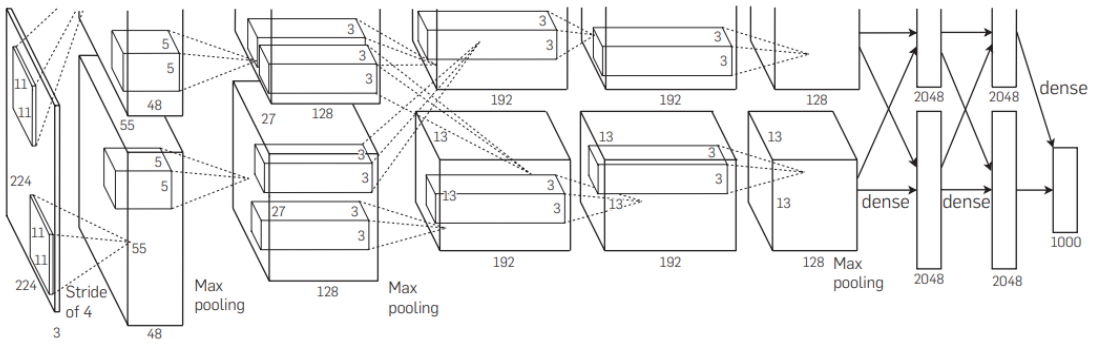


Figure 1.11 The architecture of AlexNet [16].

VGGNet [17], proposed by Karen Simonyan and her team in 2014, won second place in the ImageNet competition. VGGNet is distinguished by its continuous usage

of 3×3 convolutional kernels for convolutional operations, enhancing network depth to boost performance, and displaying superior transfer learning capabilities. It has found extensive application in the field of computer vision.

ResNet [18] was the champion in the 2015 ImageNet competition. It incorporated residual structures to address the issue of gradient vanishing in deep networks, as shown in Figure 1.12. Despite the substantial accomplishments of convolutional neural networks across various fields, the main challenge for deep neural networks currently pertains to their substantial computational costs, power consumption, and inadequate computing resources, thus rendering their deployment on edge devices a complex task.

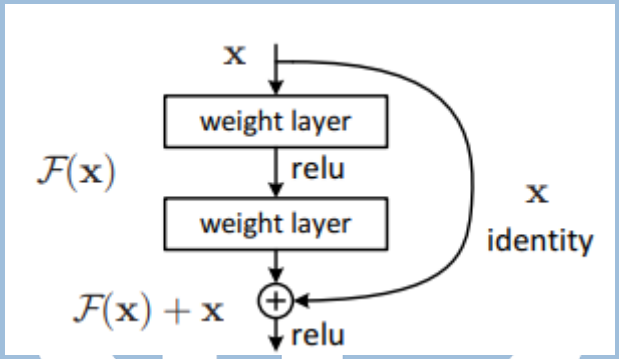
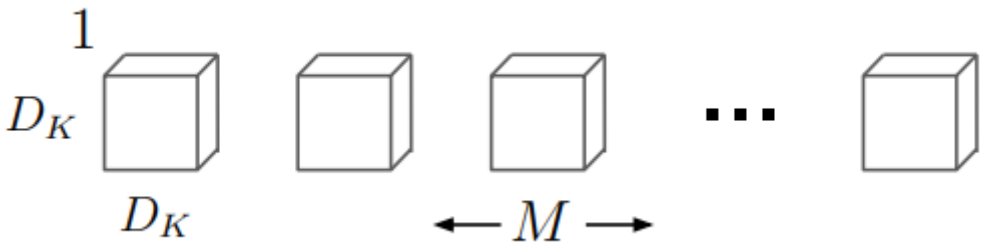


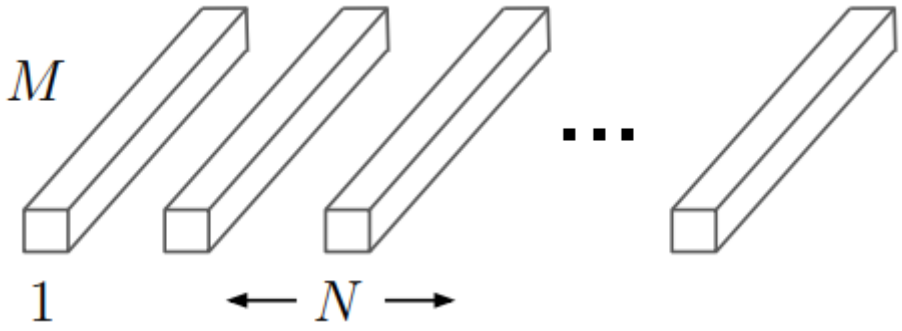
Figure 1.12 Residual block [18].

MobileNet[19], ShuffleNet[20], and SqueezeNet[21] are three lightweight models designed for mobile and edge devices, prioritizing the reduction of model size over amplifying network depth for enhanced accuracy. MobileNet, a convolutional neural network introduced by Google, curtails the number of parameters and computations using depthwise separable convolutions. These convolutions partition the standard convolution operation into depthwise and pointwise convolutions, as demonstrated in Figure 1.13. The depthwise convolution executes independent convolutions on each channel, easing the computational load. Conversely, the pointwise convolution employs a 1×1 convolutional kernel and merges features from distinct channels. These two operations can decrease the number of parameters and computations while preserving

adequate representation capability.



(a) Depthwise convolution



(b) Pointwise convolution.

Figure 1.13 Depthwise separable convolution[19].

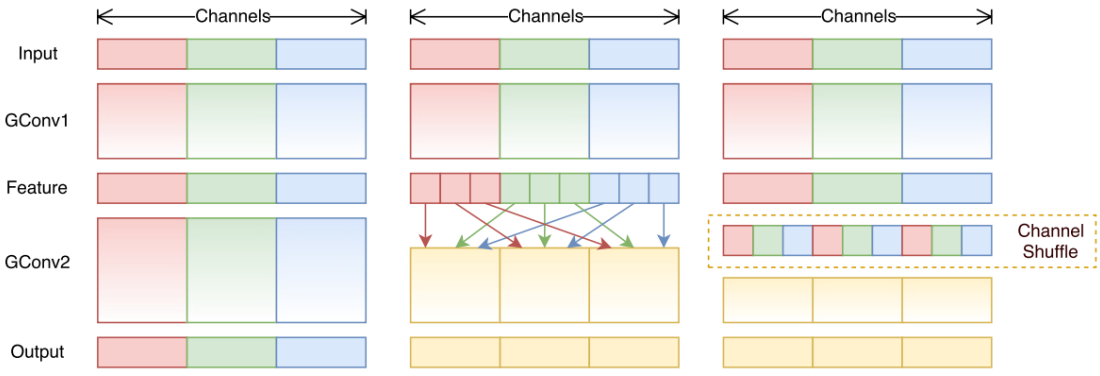


Figure 1.14 Channel shuffling [20].

ShuffleNet, a convolutional neural network introduced by the Face++ team, is specifically devised to operate on mobile and edge devices with limited computing capacity. The principal characteristic of ShuffleNet is introducing a novel operation called channel shuffling, which enhances the capacity of the network to exchange

information, as depicted in Figure 1.14.

SqueezeNet is a lightweight convolutional neural network proposed by DeepScale, UC Berkeley, and Stanford University. The network's objective is to drastically curtail the number of parameters while maintaining a performance comparable with AlexNet. The primary innovation of SqueezeNet is the inception of a novel convolutional structure termed the Fire Module. The Fire Module is bifurcated into two segments: squeeze and expand. The squeeze component employs a 1×1 convolutional kernel to reduce the number of channels in the feature map, consequently decreasing computational complexity. Conversely, the expand component utilizes 1×1 and 3×3 convolutional kernels to augment the number of channels in the feature map, thereby maintaining sufficient representative capacity, as illustrated in Figure 1.15. The combination of Fire Modules effectively reduces the parameters while upholding commendable performance.

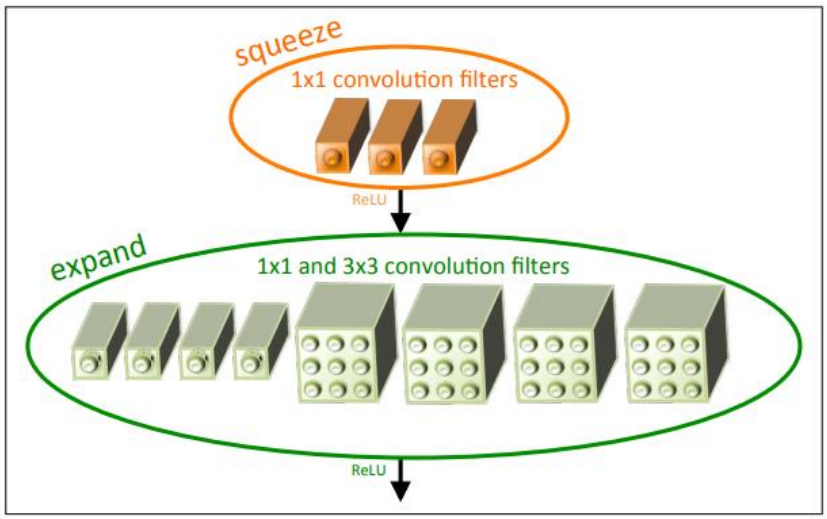
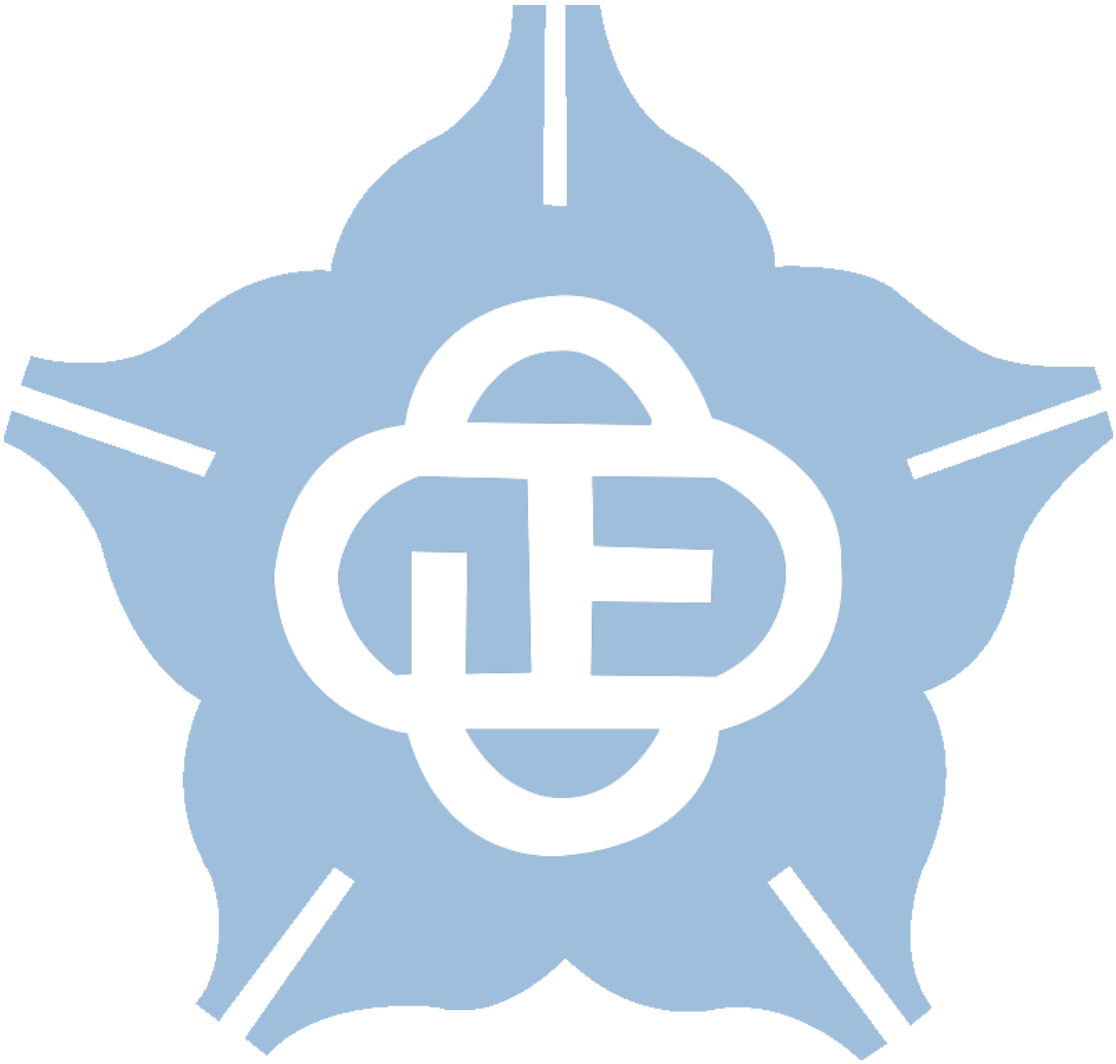


Figure 1.15 Fire module [21].

Despite the substantial accomplishments of convolutional neural networks across various fields, the main challenge for deep neural networks currently pertains to their substantial computational costs, power consumption, and inadequate computing

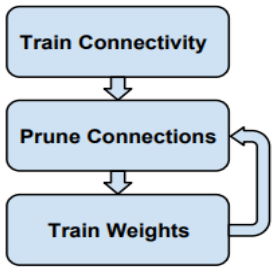
resources, thus rendering their deployment on edge devices a complex task.



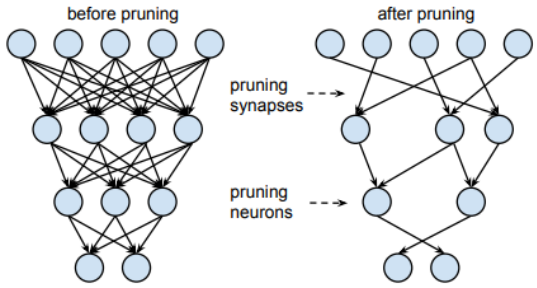
1.3 Model Compression Techniques in Neural Networks

Given the substantial size of deep neural networks, their deployment on edge devices poses significant challenges. Consequently, numerous model compression techniques have been introduced. According to [22], common techniques for condensing model size encompass network pruning, weight sharing, knowledge distillation, and early model termination.

Network pruning [23] represents a technique designed to curtail the size of a neural network by eliminating redundant neurons and weights. The process involves discarding the unnecessary elements of a pre-trained model and retraining the retained segments to ascertain that the model's performance remains unaffected to a significant degree, as demonstrated in Figure 1.16.



(a) Three-step pruning



(b) Pruned synapses and neurons before and after pruning

Figure 1.16 The network pruning technique [23].

Weight sharing realizes a model compression technique for neural networks, targeting the reduction of the number of weights and the storage requirements in a model. The fundamental concept behind weight sharing involves clustering the weight values within the neural network and substituting each weight with the representative value of its respective cluster, ensuring that weights bearing similar values share an identical representative value. This technique diminishes the model's storage space demand and can curtail computation without significantly compromising the model's performance.

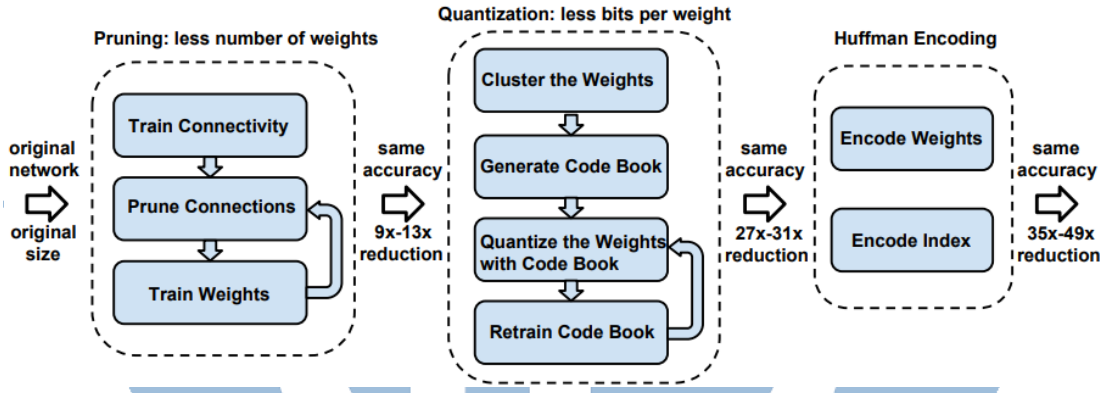


Figure 1.17 The three-stage compression process of pruning, quantization, and Huffman coding [24].

In [24], a method known as deep compression is introduced, which condenses the model through a three-step process: pruning, quantization, and Huffman coding, as depicted in Figure 1.17. Pruning consists of eliminating weights below a certain threshold from the original network and retraining the network with sparse connections until it reaches convergence. Subsequently, K-means clustering is employed, enabling weights from the same group to share an identical centroid and carry out fixed-point processing. During the weight updating process, the gradients of the same cluster are aggregated, multiplied by the learning rate (lr), and then subtracted from the shared centroid to derive the fine-tuned centroid, as illustrated in Figure 1.18. Ultimately, the

weights and indices undergo additional compression by Huffman coding.

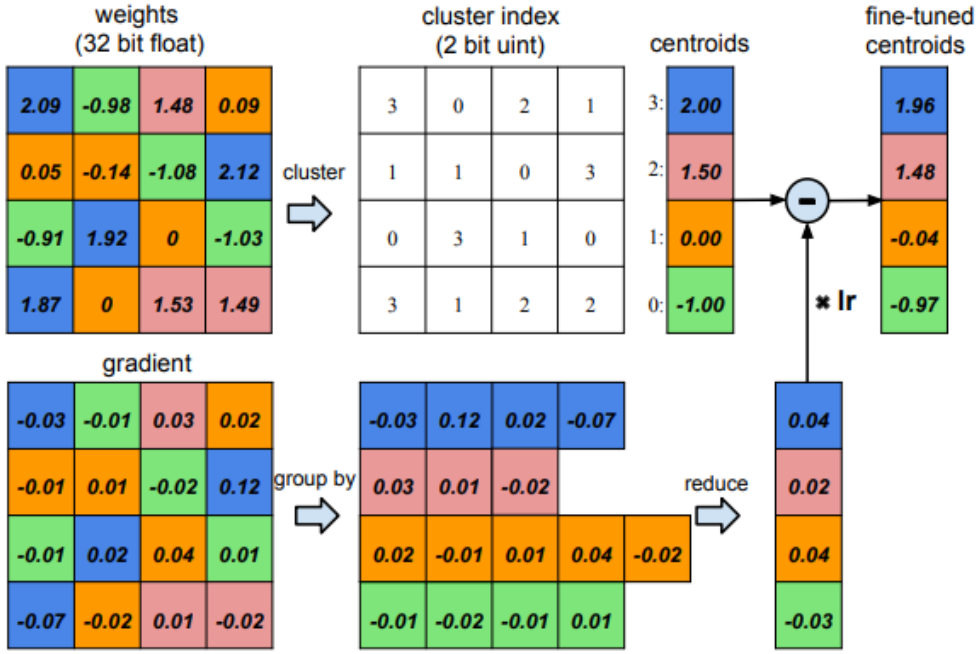


Figure 1.18 Weight sharing by K-means clustering [24].

Knowledge distillation [25] represents a method for model compression, which transfers the knowledge from a more sizable, complex model to a less substantial, simpler one. Consequently, the smaller model can utilize the features the larger one comprehends, thereby enhancing its performance.

The central concept of early model termination [26] involves integrating early exit points within the initial model. When the model has sufficient confidence to predict the input, it can expedite the termination of computation via the exit point, thereby conserving computational resources and time, as depicted in Figure 1.19. The most elementary early exit is based on a confidence threshold, which determines the possibility of an early exit. If the confidence value from the classifier's output exceeds the threshold, the model will end its operations early and deliver the prediction result. Conversely, if the confidence value falls below the threshold, the model will continue operating until the final classifier offers a prediction result.

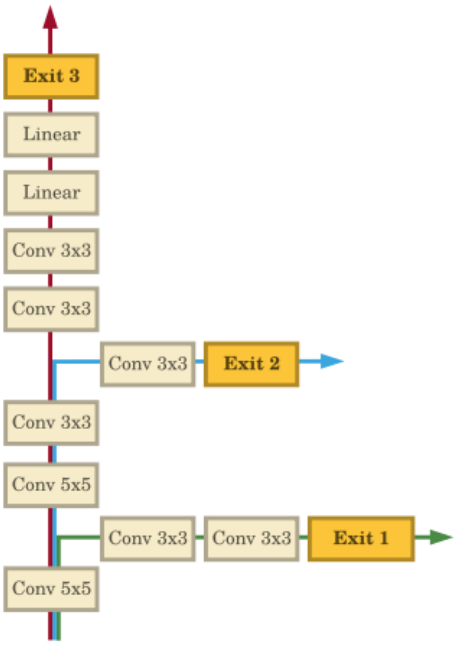


Figure 1.19 Models with two new exit points added [26].

Other model compression techniques are worth investigating beyond the frequently employed compression methods previously discussed. As deep neural networks typically grow larger and deeper, model compression has emerged as a critical technique capable of downsizing models, accelerating computation, and minimizing memory requirements. It is particularly suited to devices with limited resources.

1.4 Quantization in Neural Network

Quantization is a method employed in deep learning that aims to reduce computational and storage needs by curbing the bit count utilized to represent a model's parameters or computation precision. This technique is paramount, especially for edge devices with restricted resources.

In deep learning, model parameters are conventionally stored as 32-bit floating-point numbers, significantly demanding storage and computational resources. The fundamental principle behind quantization is transforming these 32-bit floating-point numbers into representations with smaller bit widths, such as INT8 or INT4. This modification can significantly alleviate the storage and computational needs of the model, but it may also lead to some degree of precision loss. Various techniques have been proposed to balance maintaining model performance and achieving quantization.

A Binary Neural Network (BNN) [27] is a deep neural network that utilizes binary weights and activations. Quantifying the original neural network weights and activations down to 1 bit greatly diminishes the model's demands for storage resources and enhances inference speed. The weight transformation is shown in Equation 1.7, which transposes 32-bit floating-point numbers into either +1 or -1. Nonetheless, this method incurs a considerable loss of information, inevitably leading to decreased accuracy.

$$x_b = sign(x) = \begin{cases} 1, & if\ x > 0 \\ -1, & otherwise \end{cases} \tag{1.7}$$

Ternary Weight Networks (TWN) [28] employ a weight transformation encompassing +1, -1, and 0, with storage requiring only 2-bit space. Compared with the original 32-bit floating-point representation, it can reduce the storage space by 16 times. Moreover, it offers greater ease in achieving superior accuracy compared to BNN.

The weight transformation formula for TWN is shown in Equation 1.8. Experimental results have indicated that the quantization of TWN can significantly compress and enhance the performance of many popular models.

$$w_i^t = \begin{cases} +1, if\ x > \Delta \\ 0, if\ x \leq \Delta \\ -1, if\ x < 0 \end{cases} \tag{1.8}$$

Mixed Precision Training [29], as proposed by Paulius Micikevicius and his team, is a technique that synergistically uses FP16 and FP32 to store weights, activations, and gradients during the training of a neural network. The main goal is to mitigate memory usage and computational complexity, enhancing training speed. Remarkably, this technique accomplishes this speed improvement while maintaining near-lossless accuracy.

The parameterized clipping activation for quantized neural networks (PACT) [30] is a method that introduces a trainable activation function that includes a variable parameter. This novel activation function enables the convergence of parameters during the training of the neural network, thus facilitating successful low-bit quantization. The activation function is determined by Equation 1.9, where α is a value that is dynamically adjusted during the training process based on gradient descent.

$$y = PACT(x) = 0.5(|x| - |x - \alpha| + \alpha) = \begin{cases} 0, x \in (-\infty, 0) \\ x, x \in [0, \alpha) \\ \alpha, x \in [\alpha, +\infty] \end{cases} \tag{1.9}$$

DoReFaNet [31] introduces an asymmetric quantization approach that allows the quantization of weights, activations, and gradients to any bit-width, thus enhancing the flexibility of quantization. The quantization is implemented as shown in Equation 1.10, which quantizes the input x to k bits. The method for low-bit weight quantization is represented by Equation 1.11, where the quantized weights are confined to k-bit fixed-

point numbers within the range of -1 to 1.

$$quantized_k(x) = \frac{1}{2^k - 1} round\Big((2^k - 1)x\Big) \tag{1.10}$$

$$f_w^k(r) = 2quantize_k\left(\frac{\tanh(r)}{2\max(|\tanh(r)|)} + \frac{1}{2}\right) - 1 \tag{1.11}$$

Indeed, the advancement of neural network quantization techniques has opened up new avenues for deep neural networks. Lowering computational resource demands and enhancing computational speed makes deploying deep neural networks on edge devices with limited storage space increasingly feasible. Nonetheless, it's important to remember that quantization techniques can also augment the model's prediction error. As such, the choice of the appropriate quantization technique should be informed by the specific application and scenario in question.

1.5 Resizing Images in Machine Learning

Convolutional Neural Networks (CNNs) excel at sifting through the intricacies of input images to isolate salient features, and they subsequently leverage these extracted features for various computational tasks, including but not limited to classification and prediction. An imperative stipulation when initiating the training process of a CNN is the uniformity of dimensions across all input images. Furthermore, tackling larger, high-resolution images demands a considerable allocation of computational resources, thereby underlining the cruciality of size adjustment for images fed into a CNN as a significant pre-processing measure.

Deep learning frequently resorts to interpolation techniques to rectify and standardize image size. Among these techniques, nearest neighbor interpolation, bilinear interpolation, and bicubic interpolation are more prevalent and widely applied.

The nearest neighbor interpolation technique is a paragon of simplicity within interpolation methods. It operates on a principle where each pixel value in the desired, or target, image corresponds directly to its nearest counterpart in the input image. To put it another way, this method allocates to a new pixel a value that is closest in approximation to the original pixel value. The salient merits of this technique are its computational simplicity and high processing speed. This method offers a satisfactory solution for images with minimal complexity or ones characterized by clear, definitive boundaries. However, a notable pitfall of the nearest neighbor interpolation method becomes apparent when deployed for complex images, as it can precipitate substantial image distortion.

The bilinear interpolation method is an evolved extension of the basic linear interpolation technique. Although it introduces a higher complexity level than the nearest neighbor interpolation method, it tends to deliver superior outcomes. In computing new pixel values, bilinear interpolation zeroes in on the four nearest pixels within the source image. It then cleverly estimates the value of the new pixel based on an intricate interplay between the values of these four pixels and their respective proximities to the new pixel. A key strength of bilinear interpolation is its capacity to generate more visually appealing, smoother images while significantly reducing jagged distortions. However, it poses a more complex computational challenge and requires a longer execution time than its nearest neighbor counterpart.

The most intricate method in this list, bicubic interpolation, generally offers the most commendable results. It bears structural similarities to bilinear interpolation but differentiates itself when it comes to the calculation of new pixel values. Rather than considering four, bicubic interpolation considers the sixteen closest pixels from the input image. This more expansive consideration results in a smoother output image and retains more detail from the original image. However, the downside lies in its increased computational complexity and time consumption, making it the most resource-intensive method among the discussed.

The crucial task of selecting the right image preprocessing technique cannot be overstated in the realm of CNNs. This decision must be carefully made, considering the computational complexity, potential for overfitting, and its impact on the overall training outcomes. This selection influences the overarching network architecture and has implications on other important hyperparameters, including the number of training epochs and the learning rate. Therefore, the process of determining an appropriate image preprocessing strategy is indeed a critical aspect in the orchestration of a successful deep learning workflow.

1.6 Evaluation Metrics in Binary Classification

In machine learning, binary classification is a pervasive problem in predicting one of two possible outcomes: yes or no, true or false, 1 or 0, and so on. To evaluate the performance of a model in such cases, various metrics are used, including confusion matrix, accuracy, precision, recall, and F1 score.

A confusion matrix is a specialized table used to quickly assess a machine learning algorithm's performance. It is divided into four sections: True Positive (TP), False Positive (FP), True Negative (TN), and False Negative (FN). TP refers to cases where the model predicts a positive result, and the actual input is also positive. FP denotes cases where the model predicts a positive result, but the actual input is negative. FN represents cases where the model predicts a negative result, but the actual input is positive. TN indicates cases where the model predicts a negative result, and the actual input is also negative, as shown in Table 1.1.

Table 1.1 Confusion matrix

	Actually true	**Actually false**
Predicted true	True Positive (TP)	False Positive (FP)
Predicted false	False Negative (FN)	True Negative (TN)

Accuracy is one of the most commonly used classification metrics, referring to the proportion of correct predictions made by the model compared to all outcomes. As mentioned above, the correct predictions made by the model include both True Positives and True Negatives. Therefore, the formula for accuracy can be expressed as shown in Equation 1.12.

$$(TP + TN)/(TP + FP + TN + FN) \tag{1.12}$$

Although accuracy provides a straightforward interpretation, relying solely on

accuracy to assess a model can pose challenges when dealing with imbalanced data. For instance, if all the input data is negative, a model that consistently predicts negative can achieve remarkably high accuracy. Nevertheless, this does not guarantee the model's predictive efficacy.

Precision is the proportion of true positive predictions made by the model among the inputs predicted as positive. The formula for precision can be expressed as shown in Equation 1.13.

$$TP/(TP + FP) \tag{1.13}$$

Precision evaluates the correctness of the model's positive predictions. However, it cannot provide an assessment of the model's ability to predict all true positive cases. Therefore, in such cases, it is necessary to use recall for measurement.

Recall is the proportion of positive instances predicted by the model compared to all actual positive instances, as shown in Equation 1.14.

$$TP/(TP + FN) \tag{1.14}$$

However, there is usually a trade-off between precision and recall. Improving precision may reduce recall and vice versa. In such cases, we need a metric that can consider both precision and recall simultaneously, and that is the F1 score. The F1 score represents the average of precision and recall, as shown in Equation 1.15.

$$2 * (Precision * Recall) / (Precision + Recall) \quad (1.15)$$

The F1 score helps to strike a balance between precision and recall, making it an excellent choice when the model needs to consider both precision and recall simultaneously.

In conclusion, the confusion matrix, accuracy, precision, recall, and F1 score are all essential binary classification metrics, each with its characteristics and suitable application scenarios. Therefore, when selecting metrics, it is essential to consider the specific requirements and application context of the problem to evaluate the model's performance accurately.

1.7 Motivation

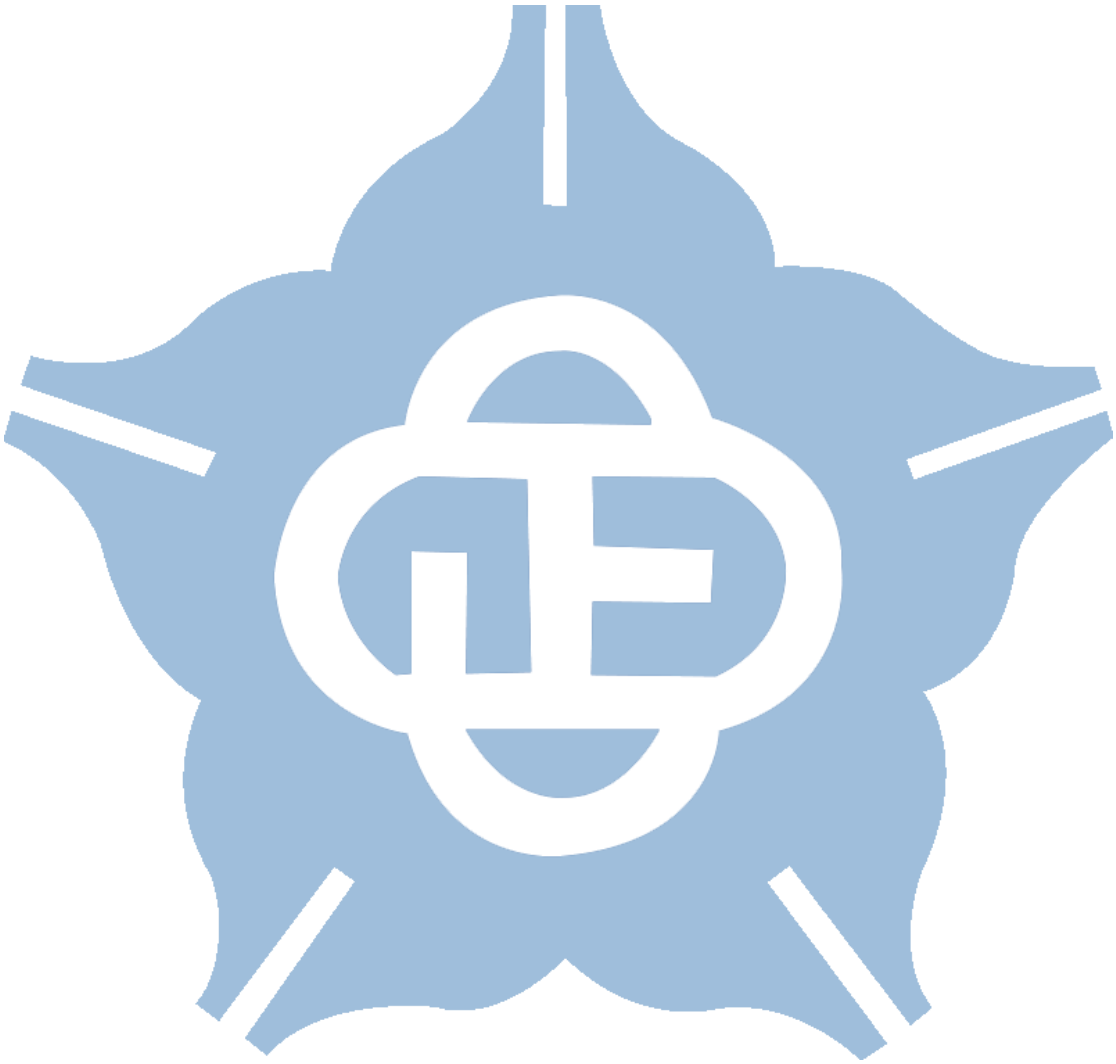
In the previous sections, we examined numerous applications of drones in modern society, where incorporating AI technology can potentially conserve substantial manpower and enable drones to carry out tasks more effectively. However, the huge computations demanded by deep neural networks pose a challenge for drones. A method that strikes a balance between performance and accuracy is necessitated.

In this thesis, we constructed a simple CNN model for a wildfire recognition dataset. An early exit point was incorporated into the model to reduce the computational load significantly. Furthermore, weight and activation quantization techniques were utilized to reduce the memory space requirements. Lastly, hardware circuits were implemented on 40nm CMOS technology to perform the computations. This work thus represents a significant step forward in developing AI-powered drone applications, demonstrating how advances in deep learning and neural networks can be successfully incorporated into drone technology for effective and efficient operation.

The contributions of this work are as follows:

1. The development of a lightweight model with early exit points designed explicitly for wildfire image recognition. This model effectively reduces the computational load of the neural network while achieving a prediction accuracy of 83%. This advancement balances resource utilization and model performance, which is critical for deploying drone AI capabilities.
2. The application of fixed-point quantization techniques to lower the memory demands of the hardware circuits used for weights and activations. These techniques were carefully chosen and applied to ensure minimal impact on CNN's accuracy. This achievement demonstrates that reducing computational resources is feasible while preserving model performance.

3. The implementation of hardware circuits on 40nm CMOS technology for efficient computation. This development represents a practical way to perform the high-level computations required by neural networks on drones, thereby enhancing the real-world applicability of AI-driven drone technology.



Chapter 2 Software Implementation

2.1 Architecture Overview

This thesis presents a method for detecting fire occurrences using UAVs. As mentioned in [6], the dataset consists of various types of fire and non-fire videos captured by UAVs. Each frame is extracted from these videos as input for classifying fire and non-fire images, as shown in Figure 1.4. A total of 39,375 frames were obtained, including 25,018 frames with fire and 14,357 without fire. The test set consists of 5,137 frames with fire and 3,480 without fire.

In building the architecture, the first step is to apply appropriate preprocessing to the input images. Choosing suitable preprocessing methods will impact the subsequent flow. Once the appropriate preprocessing methods are selected, choosing a suitable network architecture is next. The process of constructing a CNN model is illustrated in Figure 2.1, applied to binary classification data. The model's final accuracy should be at least 83% or higher. After determining the model's depth, the proposed architecture defines early exit points to reduce computational costs when deploying edge devices.

After completing the model construction, the next step is to quantize the model. Considering the limited computational resources on edge devices, reducing the model's computational demands and saving storage space is necessary. However, quantizing the model will inevitably lead to a decrease in accuracy. If the accuracy drops significantly, it will render the model impractical. Balancing model accuracy and reducing computational complexity is a crucial issue to consider.

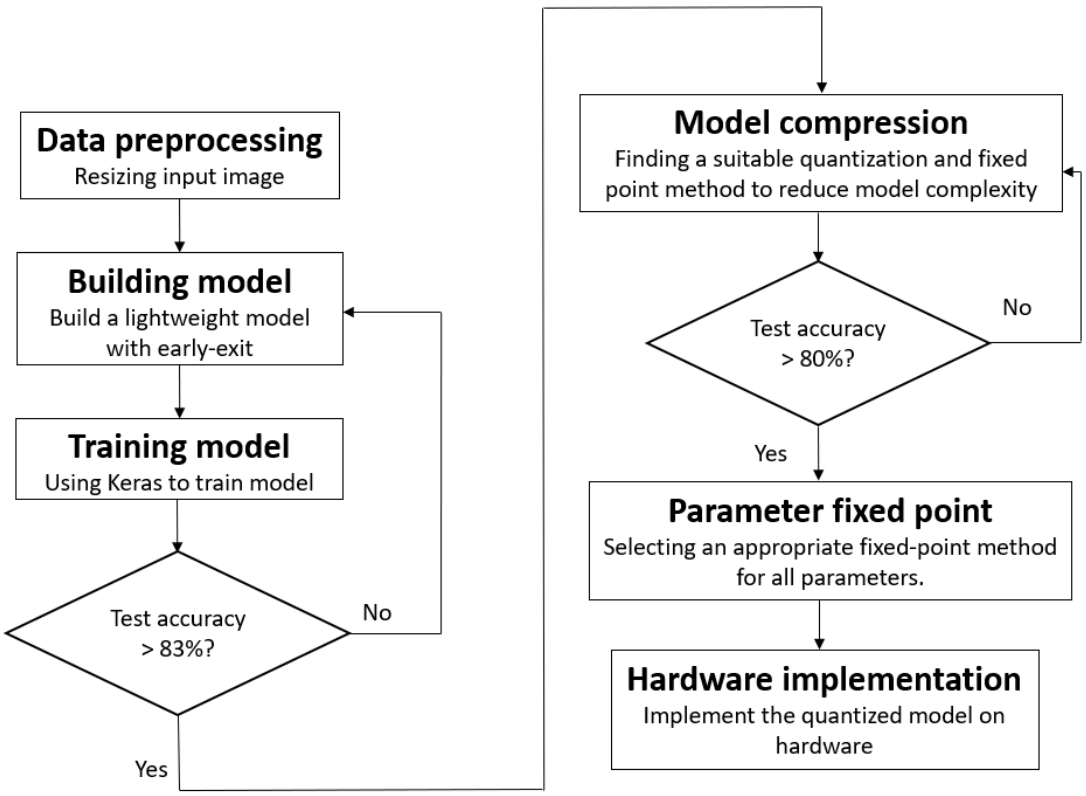


Figure 2.1 Flow chart for creating a lightweight model with an early exit.

2.2 Dataset Preprocessing

When selecting a suitable image preprocessing method, it is essential to consider whether it will significantly impact the model's accuracy and consume excessive computational resources during image processing. In Section 1.5, three commonly used image resize methods in deep learning were introduced. First, a four-layer convolutional CNN model is defined to choose an appropriate method. Then, in Table 2.1, we conducted separate tests using these three methods to observe the effect of different compression sizes on accuracy.

Table 2.1 shows that the nearest-neighbor interpolation method substantially impacts accuracy, with a significant drop in accuracy when compressing images to 32(32). On the other hand, the bilinear interpolation method and the bicubic interpolation method performed better. Considering computational complexity, the bilinear interpolation method is selected in the proposed architecture and compressed the images to 64×64.

Table 2.1 Comparing the accuracy of various compression methods at different compression sizes.

Image size	**Nearest neighbor interpolation test accuracy**	**Bilinear interpolation test accuracy**	**Bicubic interpolation test accuracy**
128 × 128	74.53%	84.70%	84.64%
64 × 64	65.32%	83.29%	83.11%
32 × 32	54.62%	75.66%	72.42%

2.3 Implementation and Analysis of CNN with Early Exit

This section introduces the proposed model architecture, model parameters, and the method for implementing early exit in the model. The feature map sizes for each layer of the model are presented in Table 2.2. The computations required at each model layer are depicted in Figure 2.2.

Table 2.2 The input and output data size of each layer.

Operation	**Input data size** (width × height × in_channel)	**Output data size** (width × height × in_channel)	**stride**
Convolution 1	64 × 64 × 3	64 × 64 × 8	1
Max-pooling 1	64 × 64 × 8	32 × 32 × 8	4
Convolution2	32 × 32 × 8	32 × 32 × 8	1
Max-pooling 2	32 × 32 × 8	16 × 16 × 8	4
Global average pooing	16 × 16 × 8	1 × 1 × 8	-
FC1	1 × 1 × 8	1 × 1 ×30	-
FC2	1 × 1 × 30	1 × 1 × 30	-
FC3	1 × 1 × 30	1 × 1 × 1	-
Convolution 3	16 × 16 × 8	8 × 8 × 16	1
Max-pooling 3	8 × 8 × 16	4 × 4 × 16	4
Global average pooing	4 × 4 × 16	1 × 1 × 16	-
FC4	1 × 1 × 16	1 × 1 × 30	-
FC5	1 × 1 × 30	1 × 1 × 30	-
FC6	1 × 1 × 30	1 × 1 × 1	-
Convolution 4	4 × 4 × 16	4 × 4 × 32	1

Max-pooling 4	4 × 4 × 32	2 × 2 × 32	4
Global average pooing	2 × 2 × 32	1 × 1 × 32	-
FC7	1 × 1 × 32	1 × 1 × 30	-
FC8	1 × 1 × 30	1 × 1 × 30	-
FC9	1 × 1 × 30	1 × 1 × 1	-

Image preprocessing begins with applying zero-padding to prevent excessive loss of image information. Subsequently, the convolutional stride is set to 1 to increase the model's receptive field. Batch normalization is then employed to enhance the model's convergence speed and apply regularization. Finally, the pooling layer is applied with a stride of 4 to reduce the size of subsequent feature maps.

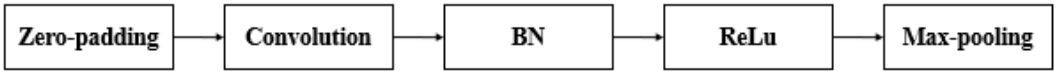


Figure 2.2 The complete operation of each layer.

After completing the convolutional operations, as illustrated in Figure 2.3, the feature map undergoes global average pooling to reduce model parameters and enhance its generalization capability. Subsequently, Dropout is applied to prevent overfitting, with a dropout rate of 50%. The resultant output is then fed into fully connected layers to accomplish image classification.

Figure 2.3 The complete operation of each classifier.

After finalizing the computations at each layer and the classifier, the early exit mechanism is applied to different layers in the network, as depicted in Figure 2.4. The advantages of early exit are discussed in Section 1.3. In the proposed network architecture, two additional early exit points are incorporated besides the final exit point, referred to as Exit1, Exit2, and Exit3. When applied in wildfire detection on drones, the choice of exit point can be based on factors such as the usage scenario, difficulty level of classification tasks, and power-saving requirements, allowing for the flexibility to switch between different exit points during model operation.

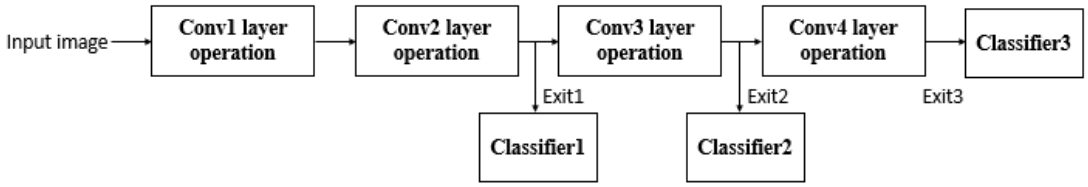


Figure 2.4 Inserting early exit points in the model.

For example, a flight strategy can be established when a drone reaches an area where wildfire detection is needed. Initially, it employed Exit1 to cover half of the planned route within the area. Subsequently, it switches to Exit2 and Exit3 to accomplish the remaining half. This approach enhances inference speed and reduces power usage without significantly compromising accuracy.

Table 2.3 presents the choices made in selecting the number of channels for the convolutional layers in the proposed architecture. The number of channels in the first convolutional layer significantly impacts the overall accuracy. Hence, Table 2.3 mainly investigates the influence of the number of channels in the first convolutional layer on the total model parameters and accuracy. Table 2.3 shows that when the number of channels is 4, the accuracy is not satisfactory. However, when the number of channels is increased to 8, a notable improvement in accuracy is achieved. Subsequent increases in channels to 16 and 32 result in only marginal improvements. Considering

computational complexity, the final choice is to use 8 channels for the first convolutional layer.

Table 2.3 The test accuracy with different channels of convolution layer1.

Method	Conv. Layer(in_channel/out_channel)				The sum of Conv. parameters	Test accuracy
	Layer1	Layer2	Layer3	Layer4		
1	3/4	4/8	8/16	16/32	6,156	73.29%
2	***3/8***	***8/8***	***8/16***	***16/32***	***6,552***	***83.11%***
3	3/16	16/8	8/16	16/32	7,344	83.57%
4	3/32	32/16	16/16	16/32	12,384	83.64%

The experimental results presented in Table 2.4 emphasize the trade-off between accuracy and inference speed. Although Exit1 may have a lower precision, using Exit2 and Exit3 provides a practical alternative with satisfactory performance. The model's behavior can be optimized by selecting the appropriate exit points based on specific application requirements, ensuring reliable and efficient wildfire detection in scenarios such as unmanned aerial vehicles or similar environments.

Table 2.4 The accuracy of the images at different exit points.

Image size	Exit1 accuracy	Exit2 accuracy	Exit2 accuracy
64 × 64	79.02%	83.04%	83.11%

Table 2.5 lists the total number of parameters in the entire neural network model. The parameters for convolutional layers and batch normalization can be calculated using the formulas presented in Equations 2.1 and 2.2, respectively. The formula for fully connected layers is shown in Equation 2.3. After global average pooling, the model's parameter count can be significantly reduced. Adding up all the parameters yields the total number of parameters in the proposed model. At Exit1, Exit2, and Exit3,

the total parameter counts are approximately 2k, 4.6k, and 11.2k, respectively.

$$Conv\ parameters = in\_channel \times kernel\ size \times out\_channel \quad (2.1)$$

$$BN\ parameters = 4 \times kernel\ size \times out\_channel \quad (2.2)$$

$$FC\ parameters = feature\ map\ size \times out\_channel \quad (2.3)$$

Table 2.5 The number of parameters of each operation.

Operation	**Number of parameters**	**Sum of parameters**
Convolution 1	3 × 3 × 3 × 8	216
Batch Normalization 1	4 × 8	32
Convolution 2	8 ×3 × 3 × 8	576
Batch Normalization 2	4 × 8	32
FC1	8 × 30	240
FC2	30 × 30	900
FC3	30 × 1	30
Convolution 3	8 × 3 × 3 × 16	1,152
Batch Normalization 3	4 × 16	64
FC4	16 × 30	480
FC5	30 × 30	900
FC6	30 × 1	30
Convolution 4	16 × 3 × 3 × 32	4,608

Batch Normalization 4	4 × 32	128
FC7	32 × 30	960
FC8	30 × 30	900
FC9	30 × 1	30
Total parameters		11,278

2.4 Weight Quantization Method

During the inference phase of a neural network, storing weights as floating-point numbers requires a significant amount of memory space. Therefore, quantizing the floating-point numbers from the original 32-bit format to lower-bit precision is necessary when executing edge-device inference operations. This work employs the DoReFaNet quantization method introduced in Section 1.4. DoReFaNet introduces a quantization function that maps the original continuous weight values to a discrete distribution between 1 and -1, as in Equation 1.11. This chapter will discuss selecting the appropriate quantization bit to balance weight accuracy and storage requirements.

Table 2.5 presents the accuracy of the same model when applying different quantization bits to each exit point. When quantizing the weights to 7 bits, accuracy is slightly decreased for all three exit points. However, when quantizing the weights to 6 bits, a more significant drop in accuracy is observed. Therefore, the final decision is to quantize the weights to 7 bits.

Table 2.6 Accuracy of weight quantization with different bits.

Bits	**Exit1 accuracy**	**Exit2 accuracy**	**Exit3 accuracy**
9	78.86%	82.71%	82.81%
8	78.62%	82.57%	82.79%
7	**78.35%**	**82.21%**	**82.69%**
6	77.21%	80.36%	80.39%

2.5 Software Result

Various metrics mentioned in Section 1.6 are utilized to assess the binary classification model's performance. Table 2.7 presents the confusion matrix of the proposed model on the wildfire classification dataset, explicitly showing the results for Exit3. After computation, the achieved metrics for the architecture following DoReFaNet quantization are as follows: accuracy of 82.69%, precision of 83.38%, recall of 87.04%, and F1-score of 85.17%.

Table 2.7 The confusion matrix of the proposed architecture.

		Predict		Accuracy
		Fire	**No Fire**	
Actual	**Fire**	4,283	854	83.38%
	No Fire	638	2,842	81.67%

In Table 2.8, the hyperparameters and loss functions employed during the model training are also listed.

Table 2.8 Hyperparameters setting ad loss function

Hyperparameter	**Value**
Batch Size	32
Train and retrain epoch	100
Learning rate	0.001

Optimization algorithm	Adam
Loss function	Cross entropy
Dropout rate	0.5

2.6 Implementing on Raspberry Pi

The proposed architecture was tested on a Raspberry Pi 3 Model B, utilizing a 1.2 GHz 64-bit quad-core ARM Cortex-A53 CPU with 1GB of memory, as illustrated in Figure 2.5. During this experiment, the neural network underwent weight and fixed-point quantization of activation values and BN parameters. The process of fixed-point quantization for activation values and BN parameters will be detailed in Sections 3.1 and 3.2, respectively.

The inference execution time for a single image was meticulously measured through repeated executions, resulting in an average execution time of approximately 3.809 seconds. Consequently, the Raspberry Pi 3 Model B can process approximately 0.26 images per second during this testing phase. This temporal metric underscores the computational efficiency of the Raspberry Pi 3 Model B within the specified experimental context.

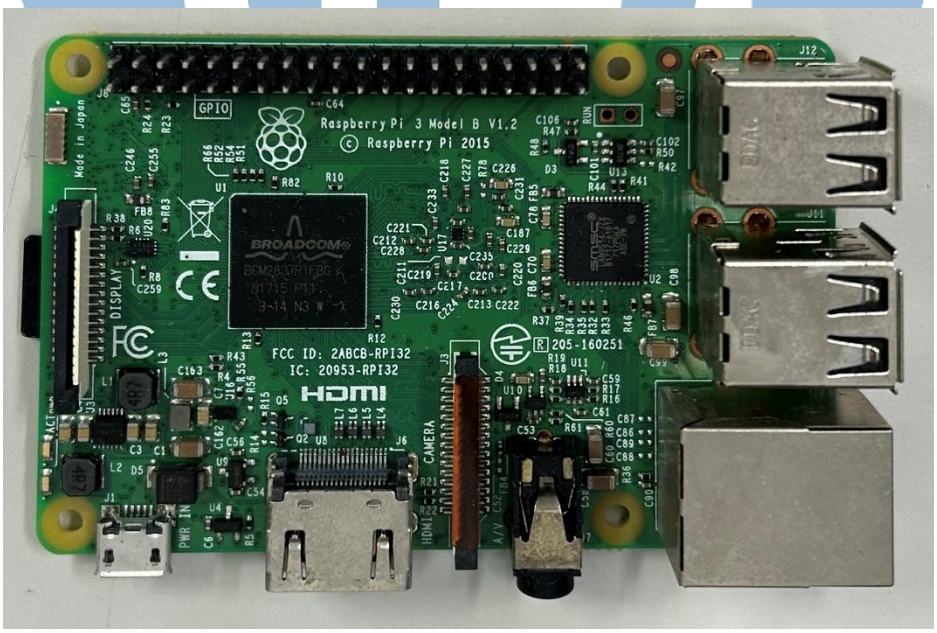


Figure 2.5 Raspberry pi 3 Model B.

Chapter3 Hardware Implementation

3.1 Fixed-Point Quantization of Activations

During the implementation of the convolutional neural network, it is necessary to write feature maps back to memory. Therefore, when performing hardware computations, reducing the number of bits for activations is essential. Testing has shown that using fixed-point quantization for activations in the proposed architecture does not significantly impact the accuracy. Hence, this section discusses the influence of different fixed-point quantization methods on accuracy since the activation functions used in the layers of the proposed architecture are ReLU, and the integer part is represented as an unsigned number with 3 bits. In contrast, the fractional part's fixed-point quantization considerably affects the neural network's accuracy.

Table 3.1 presents the accuracy of activations at different decimal precision. From the table, it can be observed that the accuracy remains relatively stable when the decimal precision is 5 bits. However, a significant decrease in accuracy is observed when reducing the decimal precision to 4 bits. Therefore, the decision is made to set the decimal precision to 5 bits, allowing activation values to be stored using only 1/4 of the memory compared to the original FP32 storage.

Table 3.1 The test accuracy comparison table for bit-width of activation.

Integer bits	**Decimal bits**	**Exit1 accuracy**	**Exit2 accuracy**	**Exit3 accuracy**
3	7	78.35%	82.16%	82.60%
3	6	78.31%	82.17%	82.57%
3	**5**	**78.29%**	**81.85%**	**81.94%**
3	4	75.37%	78.67%	78.97%

3.2 Fixed-Point Quantization of Batch Normalization

Upon completion of the model training, the four parameters of the Batch Normalization (BN) layer remain constant. To minimize memory usage for subsequent hardware implementations, quantization is applied to the BN parameters. Before quantization, the four BN parameters are simplified into two parameters, as indicated in Equations 3.1 and 3.2, respectively. These simplified equations are derived from the original BN formulas depicted in Equations 1.3 to 1.6, with μ_β representing the mini-batch mean and α_β^2 representing the mini-batch variance.

$$\gamma' = \frac{1}{\sqrt{\alpha_\beta^2}}\gamma \tag{3.1}$$

$$\beta' = -(\mu_\beta \gamma') + \beta \tag{3.2}$$

$$y_i = \gamma' x_i + \beta' \tag{3.3}$$

After the parameter simplification, the number of parameters in batch normalization is halved from 128 to 64. The numerical computations are reduced to a single multiplication and addition, as shown in Equation 3.3. This simplification facilitates the subsequent hardware implementation. The impact of quantizing the variables γ' and β' separately, focusing on the resulting error. First, the value of β' is fixed, and the bit width for quantizing γ' is adjusted. Table 3.2 presents the accuracy results after quantizing γ'.

Table 3.2 The test accuracy comparison table for the bit-width of γ'.

γ' fixed point comparison table				
Integer bits	**Decimal bits**	**Exit1 accuracy**	**Exit2 accuracy**	**Exit3 accuracy**
3	11	78.32%	81.77%	81.92%
3	10	78.35%	82.02%	81.89%
3	9	78.29%	81.83%	81.85%
3	**8**	**78.26%**	**81.76%**	**81.88%**
3	7	59.72%	60.40%	60.76%

Table 3.2 shows that the accuracy remains relatively stable across all three exit points when the fractional bit width is set to 8 or higher. However, a significant drop in accuracy occurs when reducing the fractional bit width to 7. Consequently, a fractional bit width of 8 is ultimately chosen for quantizing the variable γ'.

Table 3.3 presents the impact of quantizing β' to different bit widths on the accuracy. It can be observed that when the fractional bit width is greater than 4, there is only a slight decrease in accuracy. However, when reducing the fractional bit width to 3, both Exit2 and Exit3 experience a significant drop in accuracy.

Based on these findings, a fractional bit width of 4 is ultimately chosen for β'. Selecting an appropriate bit width allows for a balance between accuracy and resource utilization in the hardware implementation. This decision ensures that the model maintains a satisfactory level of accuracy while optimizing memory usage and computational efficiency.

Table 3.3 The test accuracy comparison table for the bit-width of β′.

β′ fixed point comparison table				
Integer bits	**Decimal bits**	**Exit1 accuracy**	**Exit2 accuracy**	**Exit3 accuracy**
3	7	78.26%	81.76%	81.92%
3	6	78.24%	81.85%	81.88%
3	5	78.18%	81.77%	81.84%
3	**4**	**78.16%**	**81.73%**	**81.82%**
3	3	77.47%	79.06%	79.04%

Batch normalization parameters will be implemented using a lookup table in the hardware implementation phase. Similarly, in Section 2.4, the quantized weights of DoReFaNet will also be stored in a lookup table, requiring fixed-point representation. The integer part of the weights can be stored using only 2 bits since DoReFaNet limits the weight range to -1 to 1. Next, the number of decimal bits to retain when storing the weights in the lookup table must be determined. The number of decimal bits has a minimal impact on the overall storage space in this case, and the decision is made to choose the number of decimal bits that minimizes the decrease in accuracy. Table 3.4 shows that the overall accuracy significantly decreases when the number of decimal bits is less than 8. Therefore, 8 bits are ultimately selected for the decimal part of the weights.

Table 3.4 The test accuracy comparison table for bit-width of weight.

Comparison for bit-width of weight				
Integer bits	**Decimal bits**	**Exit1 accuracy**	**Exit2 accuracy**	**Exit3 accuracy**
2	9	78.08%	81.68%	81.76%
2	**8**	**78.06%**	**81.63%**	**81.73%**
2	7	76.12%	78.77%	79.25%

3.3 CNN Hardware Accelerator Architecture

This section will thoroughly discuss the implementation details of the CNN with early exit points on hardware. Figure 3.1 presents the comprehensive hardware architecture, comprising the utilized computational blocks and the memory for storing weights and feature maps.

The SRAM and register files are single-port memory generated by the memory compiler in the TSMC 40nm CMOS process. They store input images and feature maps obtained after convolutional layer computations. PSUM, synthesized by a circuit synthesis tool, is a register bank that records the partial sums generated by each channel in the convolutional layer. The ROM stores the weights of the convolutional and fully connected layers.

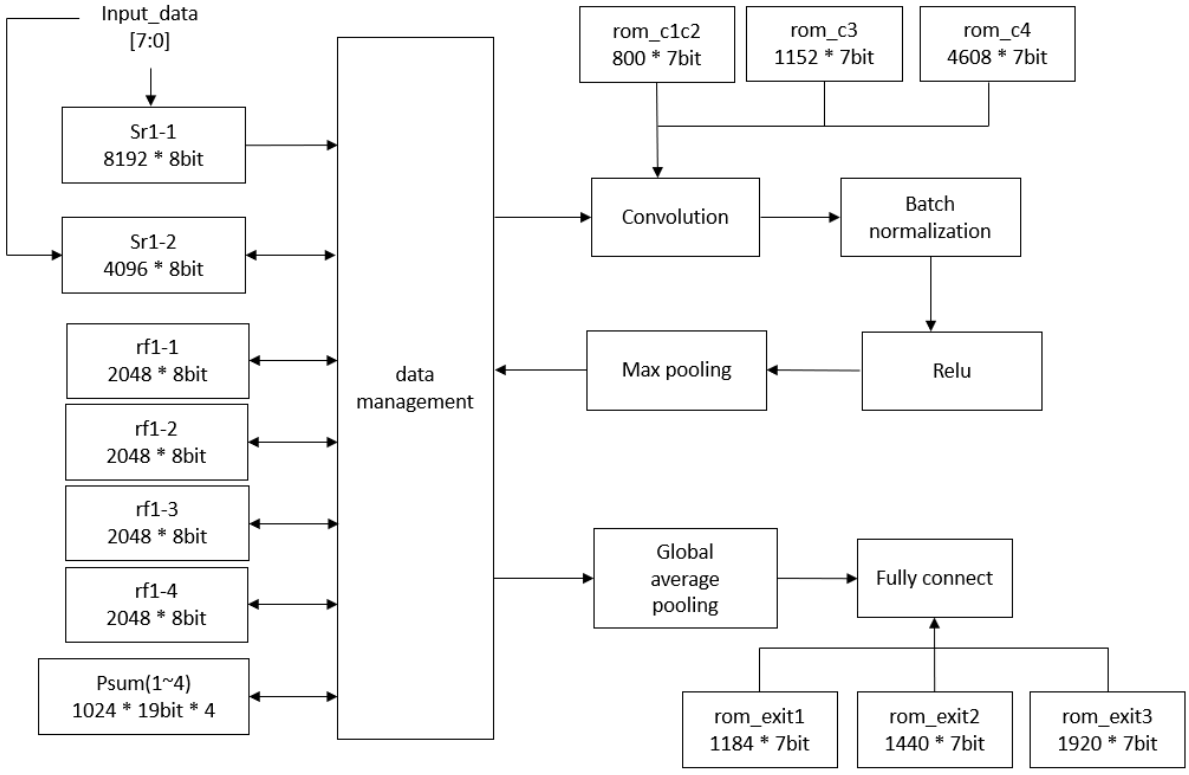


Figure 3.1 The proposed CNN model architecture diagram with an early exit.

Table 3.5 The memory usage table for each layer.

Store data	**Data size** (height × width × channel × bit-width)	**Total size(bits)**	**Data management**
Input data	64 × 64 × 3 × 8	98,306	Sr1-1, Sr1-2
Psum of layer 1	64 × 64 × 1 × 19	77,824	Psum1, Psum2, Psum3, Psum4
Feature map of layer 1	32 × 32 × 8 × 8	65,536	rf1-1, rf1-2, rf1-3, rf1-4
Psum of layer 2	32 × 32 × 1 × 19	19,456	Psum1
Feature map of layer 2	16 × 16 × 8 × 8	16,384	Sr1-2
Psum of layer 3	16 × 16 × 1 × 19	4,864	Psum1
Feature map of layer 3	8 × 8 × 16 × 8	8,192	rf1-1
Psum of layer 4	8 × 8 × 1 × 19	1,216	Psum1
Feature map of layer 4	4 × 4 × 32 × 8	4,096	rf1-2

The convolutional layer computations start from the convolution block and go through batch normalization, max pooling, and ReLU activation. The resulting feature maps are written back to the register file or SRAM. These blocks are reused in each layer's computations. After two, three, or four convolutional layers, the model's predictions are obtained through global average pooling and fully connected layers.

Table 3.5 illustrates how each memory block is utilized. The input image has a size of 64×64 pixels, in RGB format, which results in three separate channels. Given that each word is composed of eight bits, two memory blocks (Sr1-1 and Sr1-2) are needed

to accommodate the input, which occupies a total of 98.306kb. The aggregated channels in the first layer are stored in four Psum registers with a combined size of 77.824kb. After max-pooling, the downscaled feature maps are held in four register files, taking up a total of 65.536kb.

Further calculations are performed on these downscaled feature maps, so there is no need for additional storage space. Hence, these memory spaces are repurposed for later computations. When not actively used, the SRAM and register files are powered down to conserve energy. A comprehensive explanation of this process can be found in Section 3.4.

The storage capacity of Psum is divided into four separate register banks. After extensive testing, a 19-bit representation was identified as the ideal choice for maintaining accuracy while minimizing storage use. Each of these four sections has a size of 1024×19 bits. To optimize power consumption, a deliberate design decision was made to fully use all storage blocks only during the initial convolutional layer. Starting from the second convolutional layer, only a single register block is employed for channel and storage needs. The remaining three register blocks are skillfully managed with clock-gating techniques, halting their activity to meet energy efficiency goals.

In the Verilog simulation phase, the proposed architecture's required cycles are shown in Figure 3.2. It takes 12,288 cycles for the image to enter and be stored in the SRAM. When choosing Exit1 as the prediction result, it requires 232,500 cycles, Exit2 requires 274,000 cycles, and Exit3 requires 317,000 cycles. It can be observed that selecting Exit1 as the prediction result achieves nearly 1.4 times faster than Exit3, which aligns with the expectations for a lightweight model.

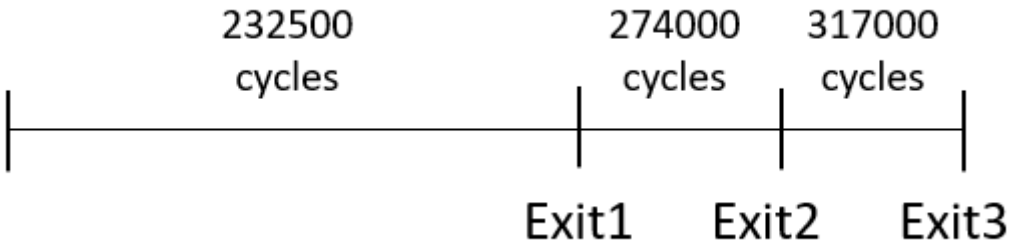


Figure 3.2 The different situations RTL-level cycles count.

3.4 Power Gating Method

The proposed architecture incorporates power gating technology to achieve the goal of low power consumption. Power gating refers to turning off the power to components such as ROM, RAM, and registers when not in use, thereby saving power. In Section 3.2, the memory space is partitioned into multiple sections, and the power to the memory is alternately turned on or off based on the required operations to reduce power consumption during idle periods.

Figure 3.3 illustrates the proposed power gating control method. Pgen1 and pgen2 are control signals for switching the SRAM on and off. Also, the signals pgen3, pgen4, pgen5, and pgen6 are control signals for the four register files. In addition, pgen7, pgen8, pgen9, pgen10, pgen11, and pgen12 are control signals for the ROM. Detailed information regarding the power switch configurations is provided in Table 3.6.

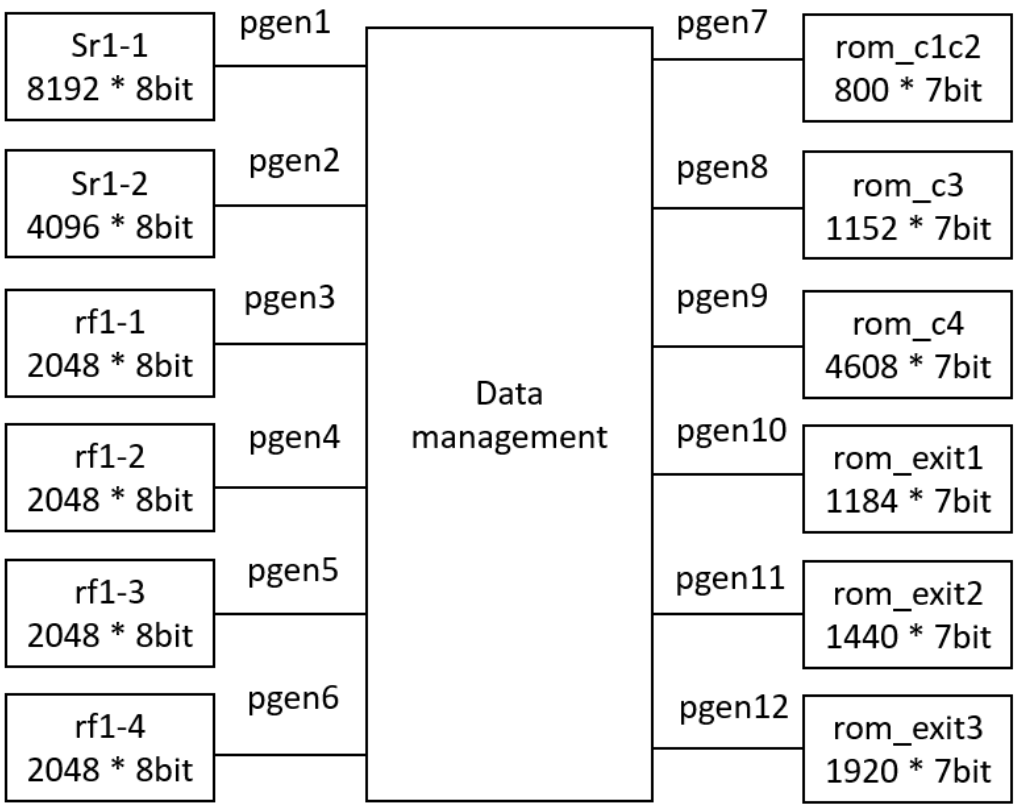


Figure 3.3 The proposed power gating control.

Table 3.6 The power state table.

layer	Power domain											
	pgen1	pgen2	pgen3	pgen4	pgen5	pgen6	pgen7	pgen8	pgen9	pgen10	pgen11	pgen12
Conv1	***On***	***On***	***On***	***On***	***On***	***On***	***On***	***Off***	***Off***	***Off***	***Off***	***Off***
Conv2	***Off***	*On*	*On*	*On*	*On*	*On*	*On*	*Off*	*Off*	*Off*	*Off*	*Off*
Exit1	***Off***	*Off*	*Off*	*Off*	*Off*	*Off*	*Off*	*Off*	*Off*	*On*	*Off*	*Off*
Conv3	***Off***	*On*	*On*	*On*	*Off*	*Off*	*Off*	*On*	*Off*	*Off*	*Off*	*Off*
Exit2	***Off***	*Off*	*Off*	*Off*	*Off*	*Off*	*Off*	*Off*	*Off*	*Off*	*On*	*Off*
Conv4	***Off***	*Off*	*On*	*On*	*Off*	*Off*	*Off*	*Off*	*On*	*Off*	*Off*	*Off*
Exit3	***Off***	*Off*	*Off*	*Off*	*Off*	*Off*	*Off*	*Off*	*Off*	*Off*	*Off*	*On*

The hardware operation flow of the entire CNN is illustrated in Figure 3.4. After the second, third, and fourth convolutional layers, the feature maps pass through three fully connected layers to predict the output results.

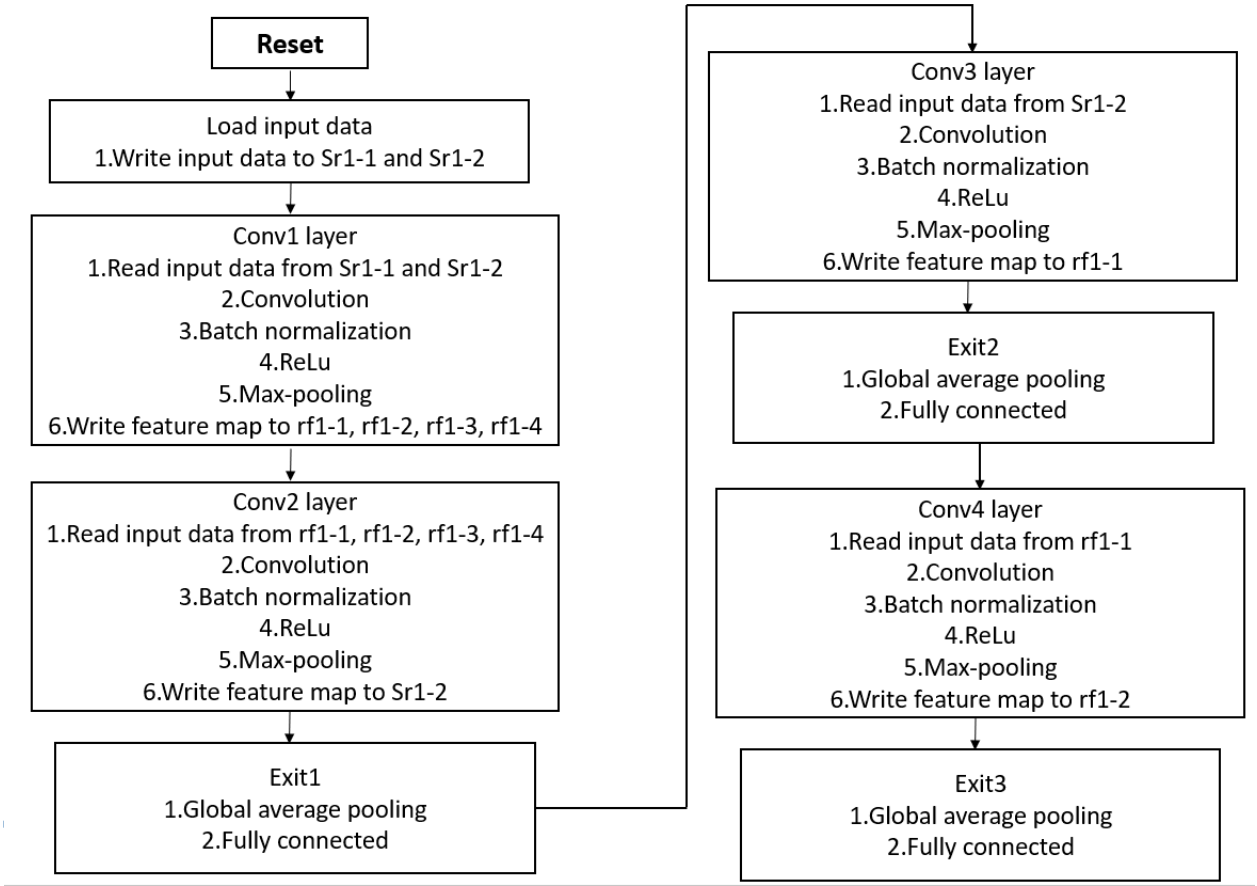


Figure 3.4 The flow chart for the CNN hardware design.

3.5 Hardware Implementation Analysis and Comparison

The proposed CNN hardware design was implemented using TSMC 40nm CMOS process. In the post-layout simulation phase, it achieved a working frequency of 100MHz. The hardware architecture, which incorporates power gating, exhibited a power consumption of 28.26mW at 100MHz. This represents an approximate 8.5% reduction in power consumption compared to the architecture without power gating, as shown in Table 3.7.

Table 3.7 Comparison with and without power gating at 100MHz and 300MHz

Method	**Total(mW)**
Power gating	28.26 @ 100MHz 112.39 @300MHz
No Power gating	30.97 @ 100MHz 117 @ 300MHz

Table 3.8 The test accuracy software to hardware in different stages.

Operation of each stage	**Exit1 test accuracy**	**Exit2 test accuracy**	**Exit3 test accuracy**
Build CNN model	79.02%	83.04%	83.11%
Weight quantization to 7-bit by DoReFaNet	78.35%	82.21%	82.69%
Convert activation to 8-bit	78.29%	81.85%	81.94%

Convert γ′ in BN to 11bits	78.26%	81.76%	81.88%
Convert β′ in BN to 7bits	78.16%	81.73%	81.82%
Verilog Register Transfer	78.02%	81.47%	81.49%

The model undergoes several steps, including quantization and fixed-point representation, from the design phase to hardware implementation. These steps may introduce changes in the model's accuracy. During RTL simulation, the intermediate computations are also constrained to fixed-point representation, leading to some quantization error. Table 3.8 presents the accuracy of the proposed architecture at each stage of completion, illustrating that even after going through the software and hardware implementation steps, the proposed architecture maintains an accuracy of over 80% at Exit3.

Table 3.9 presents the power consumption of the proposed hardware architecture at different operating frequencies. Through testing, it has been demonstrated that the proposed hardware architecture can operate at a maximum frequency of 300MHz. Equations 3.1 to 3.3 provide the computation times required for each of the three exit points, revealing that the proposed hardware architecture can achieve a range of FPS (Frames Per Second) from 9.615 to 13.16.

Table 3.9 The power consumption with different clock periods.

Clock period	**Exit1 Power consumption(mW)**	**Exit2 Power consumption(mW)**	**Exit3 Power consumption(mW)**
3.3ns = 300MHz	100.12	108.59	117
4ns = 250MHz	80.19	88.9	95.3
5ns = 200MHz	43.33	48.29	53.51

6ns = 166.67MHz	33.86	35.45	37
10ns = 100MHz	26.58	27.46	28.26
15ns = 66.67MHz	23.29	23.98	24.61

$$232500 \times 3.3\text{ns} = 767250\text{ns} \approx 0.077\text{s} \quad (3.1)$$

$$274000 \times 3.3\text{ns} = 904200\text{ns} \approx 0.090\text{s} \quad (3.2)$$

$$317000 \times 3.3\text{ns} = 1046100\text{ns} \approx 0.105\text{s} \quad (3.3)$$

The chip layout of the proposed CNN model is depicted in Figure 3.5, indicating the positions of SRAM, register file, ROM, and Psum. The chip area is determined as $1500 \times 1500\ \mu m^2$.

The implementation results of the proposed CNN architecture's hardware design are presented in Table 3.10. The power consumption is 117mW at 300MHz, the total memory size is 39 KB, and the floating-point operations (FLOP) amount to 4.28M FLOPS. These results were achieved using the TSMC 40nm CMOS process.

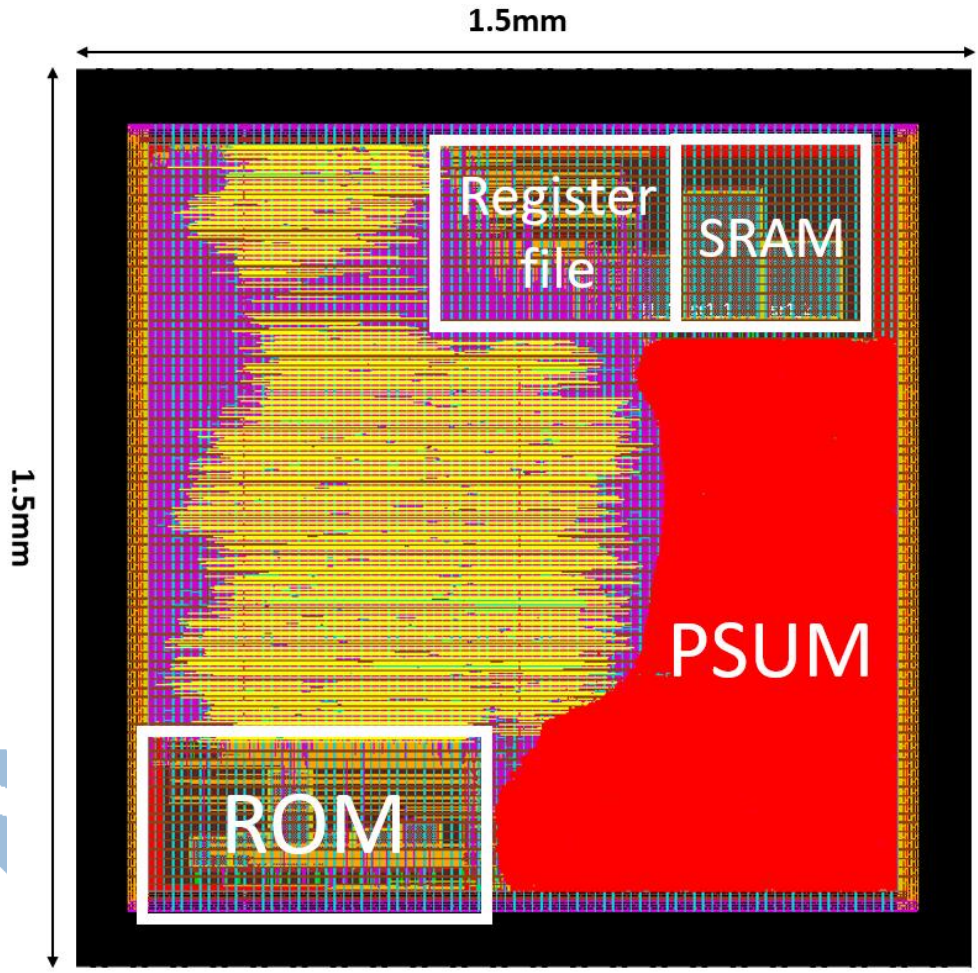


Figure 3.5 The layout of the proposed CNN hardware circuit.

Table 3.10 The CNN hardware implementation results.

	The proposed architecture
Technology	40 nm CMOS
Frequency (MHz)	300
Power (mW)	117
M FLOPS	4.28
Chip area (mm^2)	2.25
Parameters	11,278

Table 3.11 offers a comparative analysis between the proposed architecture and prior studies. In the new architecture, the initial procedure involves resizing images to a 64x64 dimension, emphasizing minimizing network parameters and overall size. Subsequently, the DoReFaNet quantization technique is applied to further decrease storage requirements and computation time for weights, all while maintaining an accuracy rate of 83%. During the hardware implementation stage, the activation values and weights are further reduced in terms of storage space through fixed-point representation.

The hardware implementation is carried out using TSMC 40nm CMOS process. At a working frequency of 300MHz, the architecture achieves an FPS range of 9.615 to 13.16, with a maximum power consumption of 117mW. The remaining hardware used in the study is the NVIDIA RTX GeForce 2080 Ti, and the provided information does not explicitly mention the power consumption. However, based on the manufacturer's official guidelines, the NVIDIA GeForce RTX 2080 Ti is suggested to have a power supply of 600W.

Table 3.11 Comparison with prior methods.

	[6] Computer Networks'21	[8] Sensors'22	[11] Forests'22	Proposed work
Architecture	Xception	DenseNet201+ EfficientB5	FT-ResNet50	2DCNN

Hardware information	NVIDIA Geforce RTX 2080ti	NVIDIA Geforce RTX 2080ti	NVIDIA Geforce RTX 2080ti	Raspberry pi3 Model B	40nm ASIC
Inference time (s)	N/A	0.018	0.055	3.809	0.077(Exit1) 0.105(Exit3) @300MHz
Dataset	The FLAME dataset	The FLAME dataset	The FLAME dataset	The FLAME dataset	
Input size	254×254	254×254	254×254	64×64	
Parameters	22.9M	50.7M	25.6M	11.2k	
Quantization method	No	No	No	DoReFaNet+Fixed-point	
Accuracy	76.23%	85.12%	79.48%	81.82%	81.49%
Power consumption	600W	600W	600W	≈2.5W	117mW @300MHz

Chapter4 Conclusion and Future Works

4.1 Conclusion

This thesis proposes a CNN model with early exit points for recognizing wildfire images using UAVs. The model architecture comprises 4 convolutional layers, batch normalization, and max pooling, and each exit point includes one global average pooling layer and 3 fully connected layers. During the model construction phase, the highest achieved accuracy was 83.11%. The weights are quantized using the DoReFaNet method, reducing the weight bit size to 7 bits. The proposed CNN model requires only 11,278 parameters.

In the RTL phase, the batch normalization parameters are simplified from 4 to 2, and the fixed-point representation of the weights is determined. In this phase, the highest accuracy achieved is 81.49%.

The proposed CNN hardware design incorporates power gating technology during the hardware implementation phase to achieve low power consumption. The design operates at a maximum frequency of 300MHz and achieves a power consumption of 117mW.

4.2 Future Works

Deploying neural networks on edge devices is a growing trend in future technological developments. However, the limited computational resources of edge devices have always been a challenge. In this thesis, a lightweight neural network is implemented, which significantly reduces computational resources through quantization. However, there is still room for improvement in terms of accuracy. Therefore, exploring how to simplify the convolutional neural network inference process is crucial while maintaining high accuracy, which is an important future research direction.

In addition to wildfire detection, UAVs offer numerous valuable applications in disaster detection, such as floods, building collapses, and road accidents. Effectively incorporating various types of disasters into implementing disaster detection is a thought-provoking and crucial issue. These UAV-based disaster detection systems provide real-time and high-resolution data, enabling rapid response and assessment during emergencies. The ability to detect and assess various types of disasters using UAVs enhances disaster management and response capabilities, making it an area of significant interest and importance for research and development in disaster management and remote sensing.

Moreover, integrating advanced technologies, such as artificial intelligence and computer vision, further improves the accuracy and efficiency of UAV-based disaster detection systems, paving the way for more effective disaster response and mitigation strategies. As UAV technology advances, the potential for enhancing disaster detection and response capabilities becomes even more promising, promising new avenues for research and innovation in disaster management and emergency response.

Reference

[1] Giovanna Castellano, Ciro Castiello, Corrado Mencar, and Gennaro Vessio, “Crowd detection in aerial images using spatial graphs and fully-convolutional neural networks,” *IEEE Access*, vol. 8, pp. 64534-64544, Apr. 2020.

[2] Pengfei Zhu, Longyin Wen, Xiao Bian, Haibin Ling, and Qinghua Hu, “Vision meets drones: A challenge, arXiv:1804.07437 [cs.CV],” *arXiv.org*, Apr. 2018.

[3] Chi Yuan, Zhixiang Liu, and Youmin Zhang, “UAV-based forest fire detection and tracking using image processing techniques,” *in Proceedings of International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 639-643, 2015.

[4] Udaya Dampage, Lumini Bandaranayake, Lumini Bandaranayake, Kishanga Kottahachchi, and Bathiya Jayasanka, “Forest fire detection system using wireless sensor networks and machine learning,” *Scientific Reports*, vol. 12, no.1, Art. no. 46, Jan. 2022.

[5] Veerappampalayam Easwaramoorthy Sathishkumar, Jaehyuk Cho, Malliga Subramanian, and Obuli Sai Naren, “Forest fire and smoke detection using deep learning-based learning without forgetting,” *Fire Ecology*, vol. 19, no. 1, pp. 1-17, Feb. 2023.

[6] Alireza Shamsoshoara, Fatemeh Afghah, Abolfazl Razi, Liming Zheng, Peter Z Ful´e, and Erik Blasch, “Aerial imagery pile burn detection using deep learning: the FLAME dataset,” *Computer Networks*, vol. 193, Jul. 2021.

[7] François Chollet, “Xception: Deep learning with depthwise separable convolutions,” *in Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1251-1258, Jul. 2017.

[8] Rafik Ghali, Moulay A. Akhlouf, and Wided Souidene Mseddi, “Deep learning and transformer approaches for UAV-based wildfire detection and segmentation,”

Sensors, vol. 22, no. 5, Art. no. 1977, Mar. 2022.

[9] Mingxing Tan and Quoc Le, "EfficientNet: rethinking model scaling for convolutional neural networks," *in Proceedings of the International Conference on Machine Learning*, pp. 6105-6114, 2019.

[10] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger, "Densely connected convolutional networks," *in Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4700-4708, Jul. 2017.

[11] Lin Zhang, Mingyang Wang, Yujia Fu, and Yunhong Ding, "A forest fire recognition method using UAV images based on transfer learning," *Forests*, vol. 13, no. 7, Art. no. 975. Jun. 2022.

[12] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929-1958, Jun. 2014.

[13] Sergey Ioffe and Christian Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *in Proceedings of International Conference on Machine Learning (ICML)*, pp. 448-456, Jul. 2015.

[14] Min Lin, Qiang Chen, and Shuicheng Yan, "Network in network, arXiv:1312.4400v3 [cs.NE]," *arXiv.org*, Mar. 2014.

[15] Yann Lecun, Leon Bottou, Yoshua Bengio, and Patrick Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, Nov. 1998.

[16] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton, "ImageNet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, no. 6, pp. 84-90, May 2017.

[17] Karen Simonyan and Andrew Zisserman, "Very deep convolutional networks for

large-scale image recognition, arXiv:1409.1556v6 [cs.CV],” *arXiv.org*, Apr. 2015.

[18] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, “Deep residual learning for image recognition,” *in Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770-778, Jun. 2016.

[19] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam, “MobileNets: Efficient convolutional neural networks for mobile vision applications, arXiv:1704.04861v1 [cs.CV],” *arXiv.org*, Apr. 2017.

[20] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun, “ShuffleNet: An extremely efficient convolutional neural network for mobile devices,” *in Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6848-6856, Jun. 2018.

[21] Forrest N. Iandola, Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer, “SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size, arXiv:1602.07360v4 [cs.CV],” *arXiv.org*, Nov. 2016.

[22] Yu Cheng, Duo Wang, Pan Zhou, and Tao Zhang, “A survey of model compression and acceleration for deep neural networks, arXiv:1710.09282v9 [cs.LG],” *arXiv.org*, Jun. 2020.

[23] Song Han, Jeff Pool, John Tran, and William Dally, “Learning both weights and connections for efficient neural network,” *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, Dec. 2015.

[24] Song Han, Huizi Mao, and William J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding, arXiv:1510.00149v5 [cs.CV],” *arXiv.org*, Feb. 2016.

[25] Jimmy Ba, Rich Caruana, “Do deep nets really need to be deep?,” *Advances in*

Neural Information Processing Systems 27 (NIPS 2014), Dec. 2014.

[26] Surat Teerapittayanon, Bradley McDanel, and H.T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," *in Proceedings of International Conference on Pattern Recognition (ICPR)*, pp. 2464-2469, Dec. 2016.

[27] Matthieu Courbariaux, Itay Hubara, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1, arXiv:1602.02830v3 [cs.LG]," *arXiv.org*, Mar. 2016.

[28] Fengfu Li, Bin Liu, Xiaoxing Wang, Bo Zhang, and Junchi Yan, "Ternary weight networks, arXiv:1605.04711v3 [cs.CV]," *arXiv.org*, Nov. 2022.

[29] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu, "Mixed precision training, arXiv:1710.03740v3 [cs.AI]," *arXiv.org*, Feb. 2018.

[30] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan, "PACT: Parameterized clipping activation for quantized neural networks, arXiv:1805.06085v2 [cs.CV]," *arXiv.org*, Jul. 2018.

[31] Shuchang Zhou, Yuxin Wu, Zekun Ni, Xinyu Zhou, He Wen, and Yuheng Zou, "DoReFa-Net: Training low bitwidth convolutional neural networks with low bitwidth gradients, arXiv:1606.06160v3 [cs.NE]," *arXiv.org*, Feb. 2018.